كلية علوم الحاسب الالي
قسم هندسة الحاسب
Computer Engineering Department

499 CPE: Project (2)
A PROJECT REPORT
PREPARED IN PARTIAL FULFILLMENT OF THE
REQUIREMENT FOR THE DEGREE OF
BACHELOR IN
COMPUTER ENGINEERING

AI-Enhanced Intrusion Detection System with Risk Assessment

Project team details omitted in this public copy

Supervisor
Dr.Omer Elsier Tayfour
may 18, 2026

BONA FIDE CERTIFICATE

Student names and signatures have been omitted from this public copy.

Dr. Mohammad Alhefdi

Head of Computer Engineering Department

Dr. Omer Elsier Tayfour



Faculty Supervisor

ACKNOWLEDGEMENT

We sincerely thank Dr. Ayman Qahmash, Dean of the College of Computer Science, King Khalid University (KKU), and Dr. Mohammad Alhefdi, Assistant Professor and Head of the Department of Computer Engineering, for their support and for providing us the opportunity to carry out this project.

We thank our project supervisor, Dr. Omer Elsier Tayfour for providing valuable guidance and help in doing the project work.

Our sincere thanks also go Eng. Mohammed Hussein ALASMRI The work would not have been possible without his kind support and help.

We also thank all faculty members of Department of Computer Engineering, College of Computer Science, KKU for their support during our study and project work.

ABSTRACT

The rapid growth of cyber threats and the increasing sophistication of modern attacks highlight the limitations of traditional signature-based intrusion detection systems such as Snort. These systems struggle to detect novel or zero-day attacks and often generate large volumes of false-positive alerts, making it difficult for analysts to prioritize incidents effectively. This project addresses these challenges by focusing on the integration of machine-learning techniques—specifically the Random Forest classifier—into the Snort detection pipeline to enhance accuracy, adaptability, and alert quality.

The project specifically investigates three aspects of the problem: improving detection accuracy, reducing false-positive rates, and introducing a risk-scoring mechanism that prioritizes alerts based on severity. To accomplish this, the proposed system combines Snort's packet-inspection capabilities with a trained Random Forest model and a real-time risk-assessment component. The system also examines deployment feasibility on a resource-constrained device such as the Raspberry Pi to ensure practical applicability in real-world environments.

Feasibility was evaluated through a review of prior research, performance analysis of ML classifiers on public IDS datasets, and hardware testing on the Raspberry Pi platform. These evaluations indicate that the project is technically achievable and provides measurable performance improvements over traditional IDS configurations.

The expected benefits of this project include higher detection accuracy, reduced analyst workload, improved response prioritization, and a deployable, low-cost IDS solution suitable for small organizations, edge networks, and academic environments.

Table of Contents

ACKNOWLEDGEMENT 2
ABSTRACT 3
Table of Contents 4
List of Tables 7
List of Figures 8

Chapter 1. :INTRODUCTION 9
1.1. Overview 9
1.1.1. Introduction to the System Concept 9
1.1.2. Block Diagram 10
1.1.3. Three-Layer System Architecture 11
1.2. Project Motivations 12
1.2.1. Growing Sophistication of Cyber Threats 12
1.2.2. Limitations of Traditional Snort-Based IDS 12
1.2.3. Evidence from Recent Research Supporting AI-Enhanced IDS 13
1.2.4. Motivation for Intelligent Risk Scoring and Prioritization 13
1.3. Problem Statement 14
1.3.1. General Problem Context 14
1.3.2. Key Gaps in Current Intrusion Detection Systems 14
1.3.3. Research Questions and Scope of the Solution 14
1.4. Project Objectives 15
1.5. Project Management 15
1.6. Background and Relevant Work 17
1.6.1. Evolution of Cyber Threats 17
1.6.2. -Based IDS vs AI/ML-Based IDS 17
1.6.3. Snort Architecture and Limitations 19
1.6.4. ML-Enhanced IDS Framework 20
1.6.5. Related Work Comparison 23
1.6.6. Research Gap 23
1.7. Significance 24

Chapter 2. SYSTEM DESIGN ALTERNATIVES 25
2.1. Design Requirements 25
2.1.1. System Needs 25
2.1.2. Derived Design Requirements 25
2.1.3. Needs–Requirements Mapping Table 26
2.2. Design Alternatives 26
2.2.1. Summary of Feasible Design Approaches 26
2.2.2. Design Alternative 1: Traditional IDS Snort Signature-Based IDS 27
2.2.3. Design Alternative 2: IDS Snort + Deep Learning IDS (NVIDIA Jetson Orin) 28
2.2.4. Design Alternative 3: IDS Snort + Random Forest + Risk Scoring 29
2.3. Design Alternatives Evaluation Criteria 30
2.3.1. Evaluation Factors 30
2.3.2. Decision Matrix 30
2.3.3. Selected Design Alternative 31
2.4. System Requirements 32

2.4.1. Functional Requirements 32
2.4.2. Hardware Requirements 33
2.4.3. Software Requirements 33
2.4.4. Non-functional Requirements 33
2.5. .Realistic Constraints 34
2.5.1. Economic Constraints 34
2.5.2. Environmental Constraints 34
2.5.3. Ethical Constraints 35
2.5.4. Health and Safety Constraints 35
2.5.5. Social Constraints 35
2.5.6. Political Constraints 35
2.5.7. Physical Constraints 36
2.5.8. Sustainability Analysis 36
2.6. Standards to be Used 37
2.7. Cost Analysis 38
2.7.1 Hardware Cost Breakdown 38
2.7.2 Software Cost Breakdown 39
2.7.3 Total Estimated Cost 39
Note: 39

Chapter 3. SYSTEM DESIGN AND IMPLEMENTATION 40
3.1. Detailed Design Description 40
3.1.1. Overall System Description 40
3.1.2. System Components Description 41
3.1.3. System Workflow 42
3.1.4. Software Module Description 42
3.2. Modeling and Simulation 43
3.2.1. Dataset Description 43
3.2.2. Simulation/Prototype/Mathematical Model 43
3.2.3. Deployment Steps 44
3.3. Hardware Implementation 44
3.3.1. Implementation Steps 44
3.3.2. Implementation Demo 45

Chapter 4. SYSTEM TESTING AND EVALUATION 46
4.1. Verification Test Description 46
4.1.1. Testing Environment Setup 46
4.1.2. Test Scenarios 46
4.1.3. Functional Verification Tests 47
4.2. Results and Analysis 47
4.2.1. Results 48
4.2.2. Analysis 49
4.2.3. Risk Scoring Evaluation 50
4.3. Discussions 50
4.3.1. Discussion of Results 50
4.3.2. System Strengths 51
4.3.3. System Limitations 51

Chapter 5. CONCLUSIONS AND RECOMENDATIONS 52
5.1. Conclusions 52
5.2. Recommendations 53

References. 54

List of Tables

Table 1.1: Snort vs. ML-Snort Performance[3] 13
Table 1.2: Gantt chart for the first semester. 16
Table 1.3:Gantt chart for the second semester. 16
Table 1.4:Comparison between signature-based IDS and AI/ML-based IDS[7][10]. 18
Table 1.5:Performance comparison of machine-learning classifiers for intrusion detection[8]. 20
Table 1.6:Comparison of Related Intrusion Detection Studies 23
Table 2.1:Mapping Between System Needs and Design Requirements 26
Table 2.2:Decision Matrix for the Three Design Alternatives 31
Table 2.3:Functional Requirements List[6][18] 32
Table 2.4:Hardware Requirements 33
Table 2.5:— Software Components and Descriptions 33
Table 2.6:Standards to Be Used 37
Table 2.7:Hardware Cost Analysis[16][17] 38
Table 2.8:Software Cost Analysis[19] 39
Table 2.9:Total System Cost Summary 39
Table 3.1: System Components 41
Table 3.2: stages for project 44
Table 3.3:Scenarios Attack with Risk Level 47
Table 3.4:Functional Verification Tests 47
Table 3.5: examples of attack classifications and their assigned risk levels. 50

List of Figures

Fig 1.1:General network layout showing the position of the IDS within the enterprise network....9
Fig 1.2:Block Diagram.....10
Fig 1.3:Three-layer architectural.....11
Fig 1.4:Organizational cyber risks ranked – 2025 12
Fig 1.5:Performance Comparison: Snort vs. ML-Enhanced Snort [3].....13
Fig 1.6:Waterfall Model of the project.....15
Fig 1.7:Global Cybersecurity Threat Trends.....17
Fig 1.8: Snort Signature Architecture[12].....19
Fig 1.9:Snort with ML-Enhanced Architecture.....20
Fig 1.10:Random Forest Detection Accuracy Comparison[13].....21
Fig 1.11:False Positive Rate Comparison for Random Forest and Snort IDS[13] 22
Fig 1.12:CPU Utilization and Processing Latency for Random Forest vs. Snort IDS[13]. 22
Fig 2.1: Design Alternative 1 27
Fig 2.2: Design Alternative 2 28
Fig 2.3: Design Alternative 3 29
Fig 3.1: System Workflow.....40
Fig 3.2: CICIDS2017 Attack Type 43
Fig 3.3: Demo Web Attack 45
Fig 3.4: Demo Dos Attack.....45
Fig 4.1:Results of model 49

Chapter 1. :INTRODUCTION

1.1. Overview

1.1.1. Introduction to the System Concept

In the past years, due to the boom in digital networks and connected devices, the volume and complexity of the cyber threat arena have multiplied. Most traditional Intrusion Detection Systems (IDSs) rely on predefined signatures or manual rules, thus rendering them impotent against new or unknown forms of attack[1].

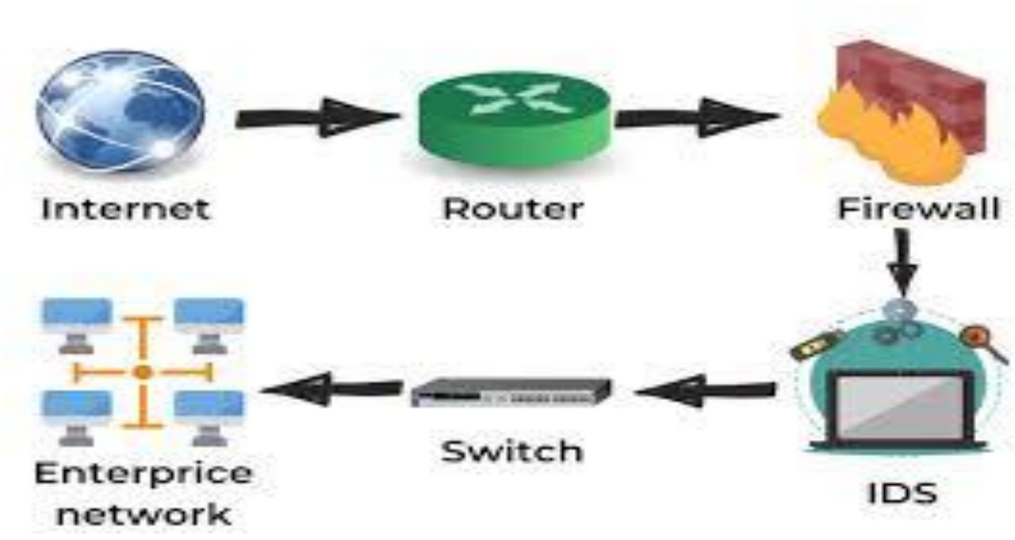


Fig 1.1:General network layout showing the position of the IDS within the enterprise network

This project proposes an AI-Enhanced Intrusion Detection System (IDS) integrated with a Risk Assessment module that intelligently evaluates and prioritizes detected threats. Incident analysis, based on machine-learning algorithms applied to network traffic patterns, identifies abnormal behaviors and risk score incidents based on potential impact. The architecture combines real-time monitoring, intelligent filtering, and automated alerting supporting early threat mitigation.

This project utilizes Python, Snort, and machine learning algorithms specifically the Random Forest classifier—to build a scalable and adaptive intrusion detection and risk-assessment system designed to enhance the security of modern digital infrastructures.

1.1.2. Block Diagram

The following block diagram presents a simplified view of how the system operates. It highlights the main functional components, including Snort-based detection, feature extraction, machine-learning classification using Random Forest, and the final risk-assessment stage that delivers alerts to the dashboard.

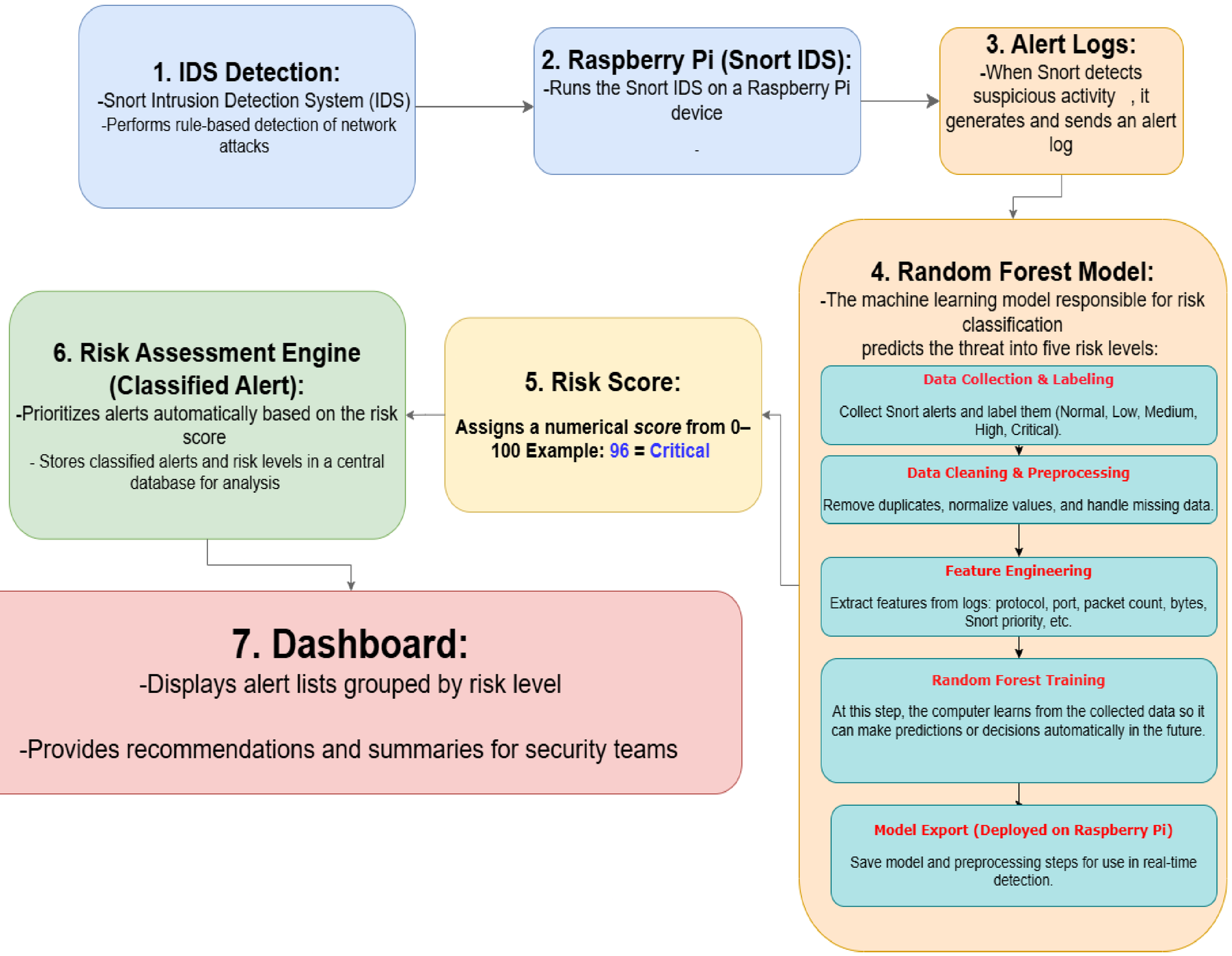


Fig 1.2:Block Diagram

1.1.3. Three-Layer System Architecture

The system is structured into three logical layers: ***data collection, intelligent processing, and response***.

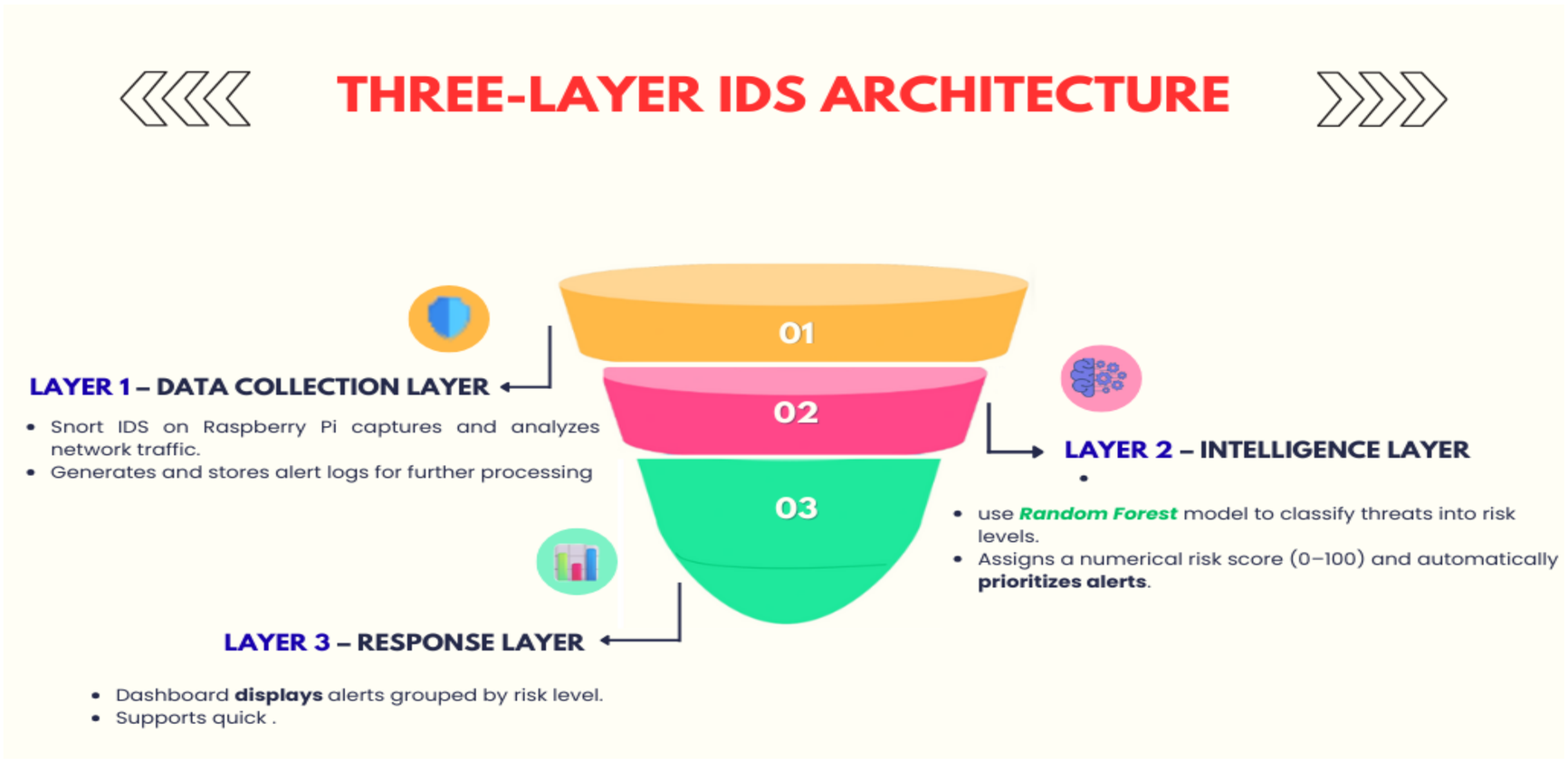


Fig 1.3:Three-layer architectural

Layer 1: ***Data Collection Layer***

This layer is responsible for capturing and monitoring raw network traffic. Snort, deployed on the Raspberry Pi, analyzes incoming packets, applies rule-based detection, and generates alert logs whenever suspicious or abnormal behavior is detected.

Layer 2: ***Intelligence Layer***

In this layer, machine-learning techniques—specifically the Random Forest classifier—process the alerts generated by Snort. Extracted features such as protocol type, packet counts, and priority levels are analyzed to classify threats into predefined categories. This layer transforms raw alert information into meaningful risk insights using intelligent decision-making models.

Layer 3: ***Response Layer***

The final layer assigns a numerical risk score to each classified incident and prioritizes alerts based on severity. The results are displayed on a dashboard interface that supports quick evaluation and decision-making by security teams. This layer ensures that high-risk threats receive immediate attention while lower-risk events are documented for later analysis.

1.2. Project Motivations

1.2.1. Growing Sophistication of Cyber Threats

The rapid development of modern cyberattacks—such as ransomware, phishing campaigns, zero-day exploits, and large-scale data breaches—has created an increasingly complex threat landscape for organizations. As network infrastructures expand and traffic volume rises, traditional security tools are becoming less effective at identifying advanced and previously unseen attacks. This escalation in attack sophistication motivates the need for a more adaptive and intelligent security mechanism capable of analyzing dynamic network behavior and identifying anomalies in real time[2].

This global trend reinforces the need for adaptive security systems that go beyond static

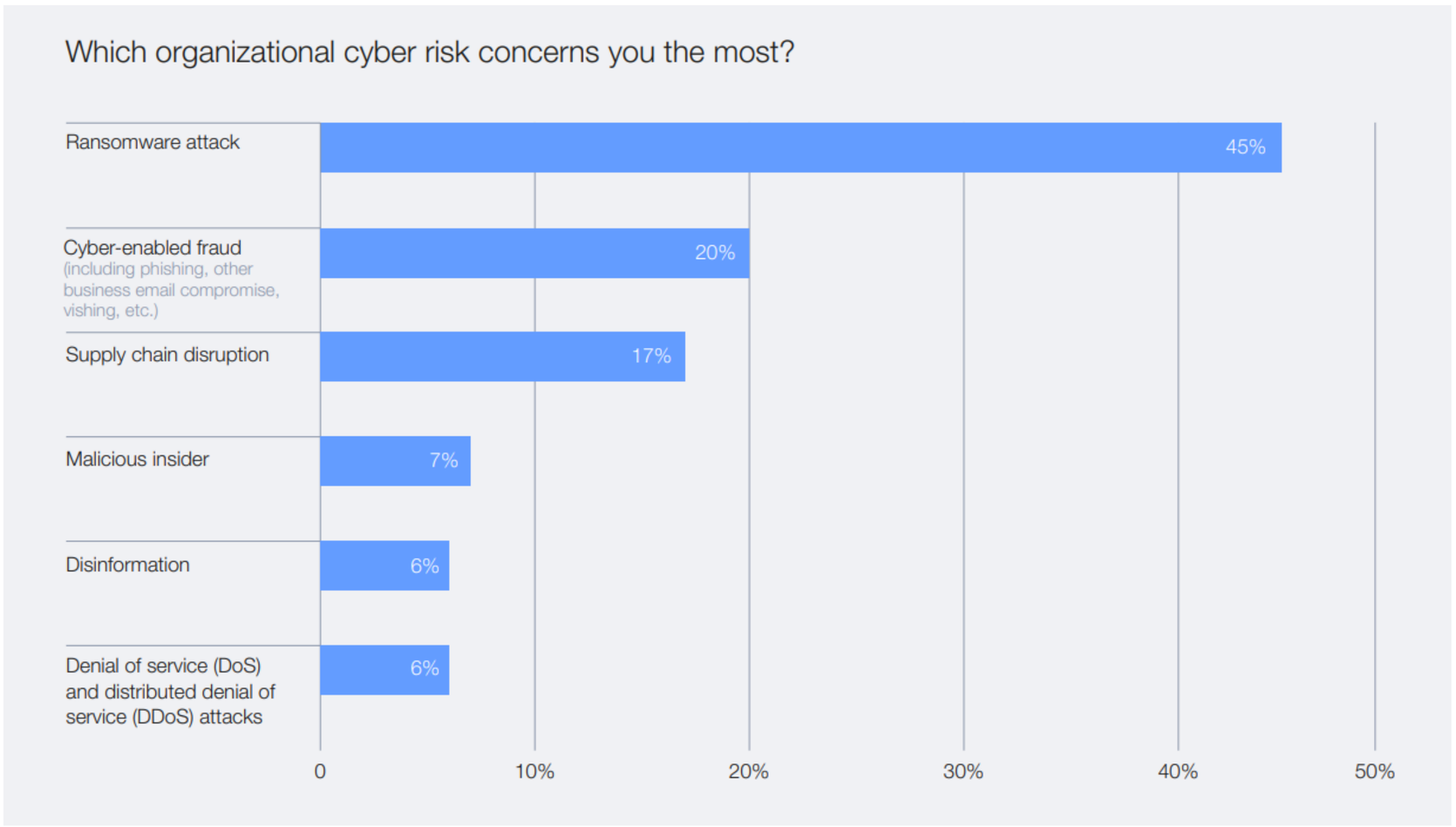


Fig 1.4:Organizational cyber risks ranked – 2025

signatures, as highlighted in recent global threat assessments such as the WEF Global Security Outlook 2025. These findings support the motivation to develop an AI-enhanced IDS capable of analyzing network behavior and detecting emerging threats more effectively.

1.2.2. Limitations of Traditional Snort-Based IDS

Snort remains one of the most widely used open-source intrusion detection systems due to its efficiency in rule-based traffic analysis. However, its static signature-matching mechanism limits its performance against novel or rapidly evolving attacks. Traditional Snort deployments often suffer from high false-positive rates, limited adaptability, and difficulty handling large or heterogeneous datasets. These limitations cause alert fatigue for security teams, reduce operational efficiency, and create gaps where genuine threats may go undetected[3][4].

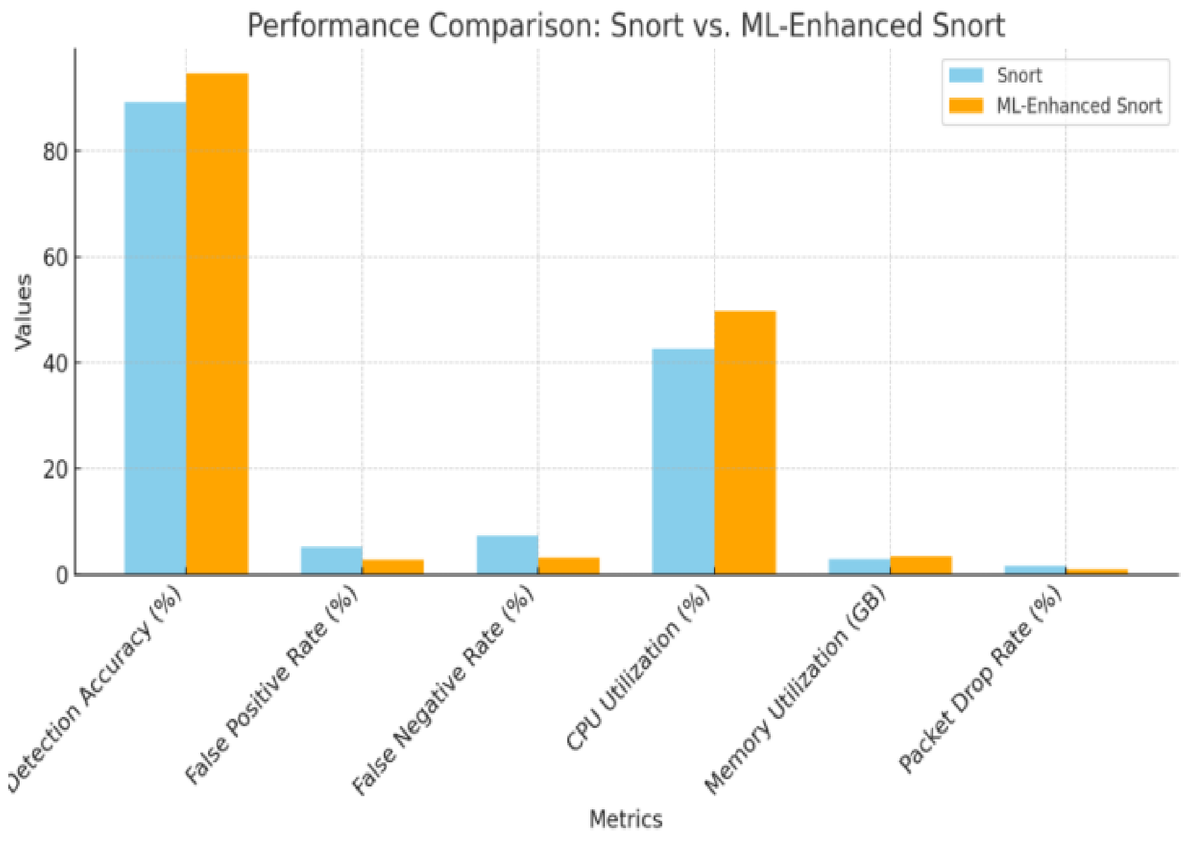


Fig 1.5:Performance Comparison: Snort vs. ML-Enhanced Snort [3].

1.2.3. Evidence from Recent Research Supporting AI-Enhanced IDS

Recent studies provide strong empirical evidence that integrating machine learning with Snort significantly improves intrusion detection performance. The AIML-Snort framework demonstrated remarkable enhancements across key performance metrics, outperforming baseline Snort configurations in realistic network environments.

Kumar et al. (2025) compared traditional Snort with an ML-enhanced Snort architecture using Random Forest and Neural Network models. Their results showed:

Table 1.1: Snort vs. ML-Snort Performance[3]

S.No	Metric	Snort	ML-Enhanced Snort
1	Detection Accuracy (%)	89.2%	94.6%
2	False Positive Rate (FPR %)	5.1%	2.7%
3	False Negative Rate (FNR %)	7.3%	3.1%
4	CPU Utilization (%)	42.5%	49.8%
5	Memory Utilization (GB)	2.8	3.4
6	Packet Drop Rate (%)	1.5%	0.9%

These results strongly motivate the integration of AI techniques into IDS systems. By employing machine-learning algorithms such as Random Forest, IDS tools gain the ability to detect unseen threats, reduce false alerts, and improve accuracy without relying solely on predefined signatures[3].

1.2.4. Motivation for Intelligent Risk Scoring and Prioritization

In modern cybersecurity operations, not all alerts have the same severity or operational impact. Traditional IDS platforms generate large volumes of alerts, making it difficult for analysts to distinguish between low-priority and high-priority incidents. This often results in alert fatigue, delayed response, and misallocation of resources.

Introducing an automated risk-assessment layer allows the system to assign a numerical risk score (0–100) to each detected incident. This supports prioritization, allowing high-risk alerts—such as critical attacks—to be addressed first. Incorporating a risk-scoring model into the IDS pipeline reduces analyst workload, improves incident-response timing, and aligns with global best practices in security analytics and threat management.

1.3. Problem Statement

The identified challenges and research questions form the basis for the following problem statement, which outlines the core objectives and focus areas of this study.

1.3.1. General Problem Context

Modern enterprise networks are experiencing rapid growth in both scale and complexity, accompanied by increasingly sophisticated cyberattacks that routinely bypass traditional security mechanisms[5]. Conventional intrusion detection systems, particularly signature-based platforms such as Snort, rely heavily on predefined rules to identify malicious activity[6]. While effective against known attacks, these systems struggle to detect emerging or previously unseen threats. Additionally, they generate large volumes of alerts, making it difficult for security teams to distinguish genuine intrusions from benign activity. This creates operational inefficiencies, delays in incident response, and exposes organizations to significant security risks[6][7].

1.3.2. Key Gaps in Current Intrusion Detection Systems

Existing IDS solutions suffer from several limitations that directly impact their effectiveness. Signature-based detection lacks adaptability, resulting in high false-positive and false-negative rates, especially when facing zero-day or polymorphic attacks. Moreover, current systems do not provide a mechanism to assess or prioritize the severity of detected events, causing critical alerts to be buried among numerous low-risk notifications. These gaps highlight the need for an intelligent, adaptive, and context-aware detection framework capable of analyzing network behavior rather than relying solely on static signatures[7][8].

1.3.3. Research Questions and Scope of the Solution

This project focuses on answering the following research questions:
RQ1: Can combining Snort with a machine-learning model (Random Forest) improve the accuracy of detecting network attacks?
RQ2: Will the use of machine learning help reduce the number of false-positive alerts generated by Snort?
RQ3: Does adding a risk-scoring module improve the prioritization of alerts and speed up incident response?
RQ4: How can a machine-learning-based risk scoring process support security teams in making faster and more informed intrusion response decisions?
These questions define the scope of the project and guide the system's design and evaluation.

1.4. Project Objectives

The primary objective of this project is to design and implement an intelligent Intrusion Detection System (IDS) that enhances traditional detection methods using ML techniques. The project is organized around the following particular goals in order to accomplish this goal:

1) **To develop an intelligent intrusion detection model using Machine Learning techniques** to improve the system's ability to analyze network behavior and identify malicious activities more accurately than traditional signature-based IDS approaches.
2) **To integrate a risk-assessment module for classifying alerts into graded severity levels**—Normal, Low, Medium, High, and Critical—by employing the Random Forest classifier to support efficient prioritization and incident handling.
3) **To combine Snort IDS with a Random Forest-based detection model** in order to enhance overall detection accuracy, reduce false positives, and provide a more adaptive and data-driven security monitoring capability.
4) **To design a real-time visualization dashboard** that displays risk levels, confidence scores, and key system performance indicators, enabling rapid interpretation of alerts and supporting timely decision-making by security analysts.

1.5. Project Management

The project adopts the Waterfall Methodology for organization. Under the Waterfall Method, development is scheduled in stages that are systematic and sequential. Each phase relies on the previous one for its deliverables. Thus, it has proper documentation and quality control. The model clearly lays down the steps for development .

Our project follows the Waterfall model, where each stage's output serves as the next stage's input. This model is simple, clear, and easy to follow.

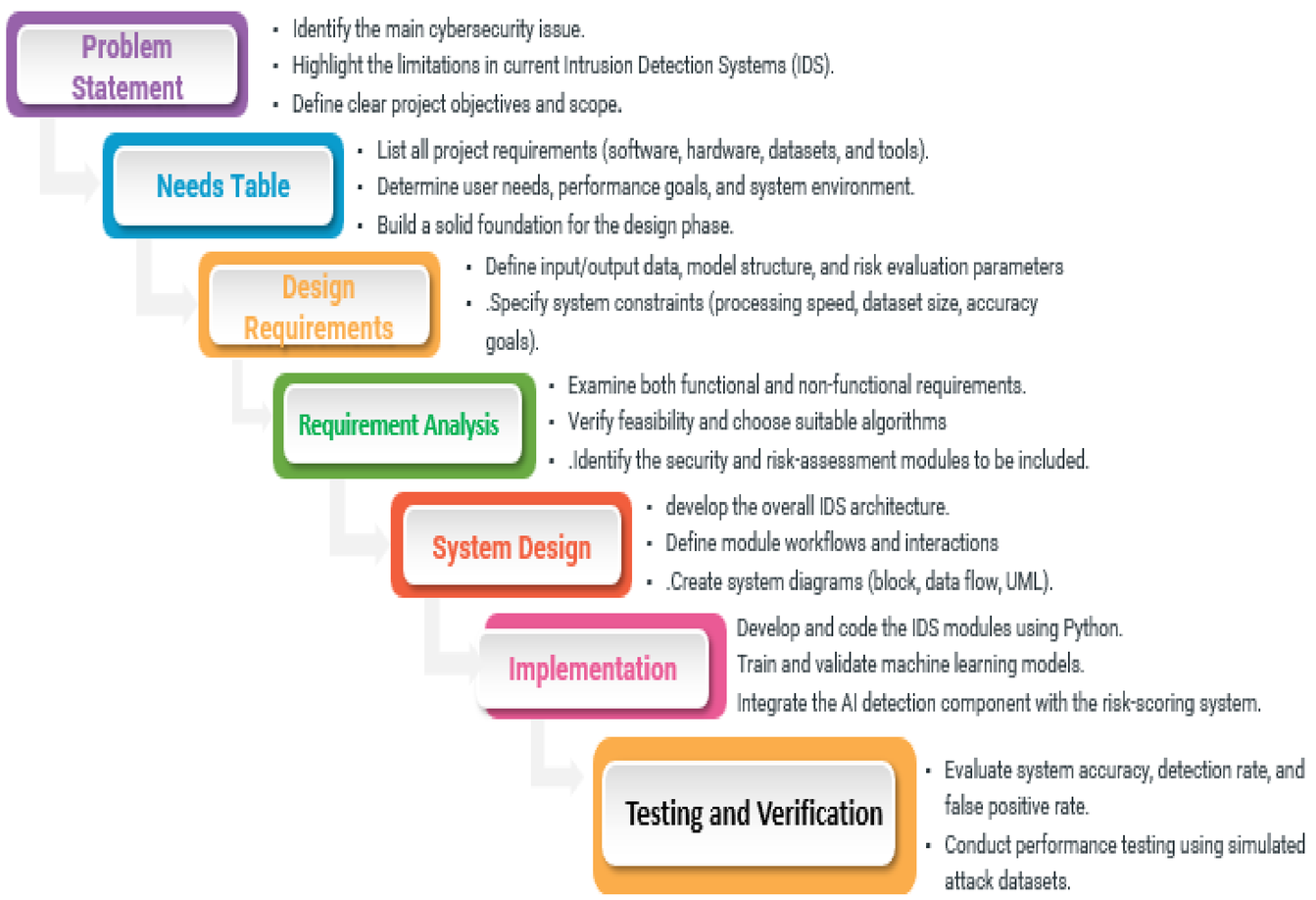


Fig 1.6:Waterfall Model of the project.

Table 1.2: Gantt chart for the first semester.

.

	Time (week)															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Selection of project and team formation																
Title selection and approval																
Literature review on IDS																
Needs table and Design requirements																
Design alternatives																
Block diagram																
Chapter 1 submission																
Hardware and software components																
Standards and constraints																
Chapter 2 submission																

Table 1.3:Gantt chart for the second semester.

	Time (week)															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
IDS Project Planning and Requirement Analysis																
Snort Installation and Configuration																
Dataset Collection and Alert Extraction																
Data Preparation and Preprocessing																
Random Forest Model Development																
Attack Classification and Risk Scoring																
Chapter 3 Submission																
System Testing and Performance Evaluation																
Results Analysis and Discussion																
Conclusions and Recommendations																
Chapter 2 submission																

1.6. Background and Relevant Work

This section provides the necessary technical background and a review of relevant work to establish the context of the problem, highlight current limitations in intrusion detection systems, and demonstrate the need for the proposed Snort–ML integration.

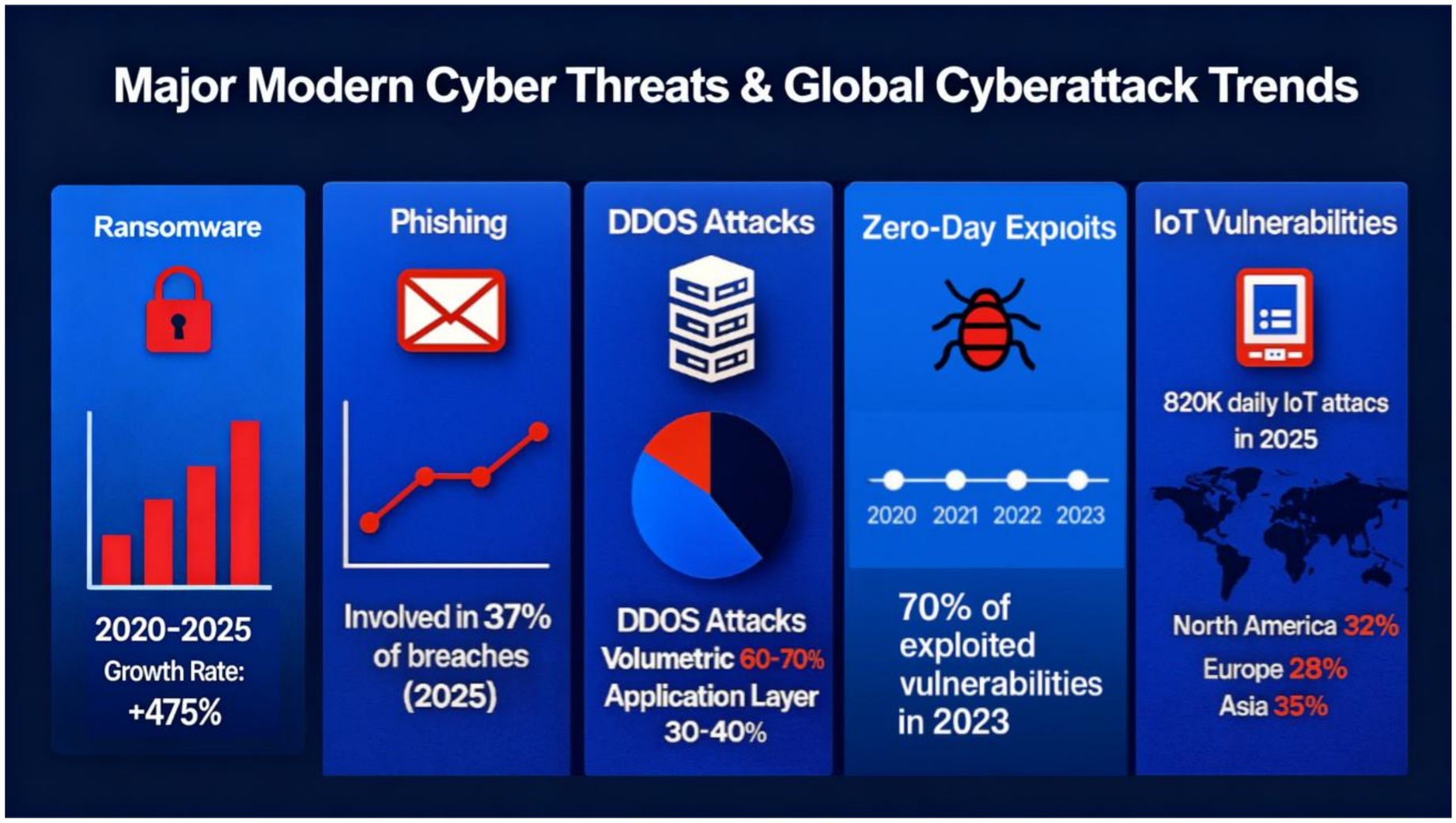


Fig 1.7:Global Cybersecurity Threat Trends

1.6.1. Evolution of Cyber Threats

The landscape of cyber threats has undergone significant transformation in recent years, with clear growth in frequency, sophistication, and diversity of attack techniques. Recent threat-intelligence statistics report that ransomware damage costs have multiplied between 2020 and 2025, phishing remains involved in roughly one-third of documented breaches, a majority of DDoS attacks are volumetric while a smaller but critical portion targets the application layer, around 70% of exploited vulnerabilities in 2023 were linked to previously unknown flaws, and organizations now face hundreds of thousands of IoT-focused attacks per day across major regions such as North America, Europe, and Asia, according to consolidated figures from ENISA, Cloudflare Radar, Fortinet, DeepStrike, and DemandSage industry reports for 2024–2025 [9].

1.6.2. -Based IDS vs AI/ML-Based IDS

Intrusion Detection Systems (IDS) are fundamental to cybersecurity, serving as a defense mechanism against malicious activities within networks. Traditional signature-based IDS rely on predefined attack signatures, which makes them effective against known threats but weak in detecting zero-day attacks and adapting to evolving techniques. In contrast, AI/ML-based IDS

utilize machine learning to analyze behavioral patterns, offering stronger adaptability and improved detection of emerging threats. *Table 1.3* presents a comparative overview of these two approaches, highlighting differences in detection methods, adaptability, computational cost, and examples[3][7][10].

Table 1.4:Comparison between signature-based IDS and AI/ML-based IDS[7][10].

Feature	Signature-Based IDS	AI/ML-Based IDS
Detection Method	Predefined signatures	Learned behavior patterns
Zero-Day Detection	Weak	Strong
Adaptability	Low	High
Computational Cost	Low	Moderate
Example	Snort	Random Forest-based IDS

Signature-based intrusion detection systems form the foundation of many traditional cybersecurity defenses. They operate by comparing network traffic against a predefined set of signatures—specific patterns known to represent malicious activity. This approach provides high accuracy for previously documented attacks; however, it depends entirely on the continuous update of signature databases. As a result, signature-based IDS struggle to detect new, modified, or zero-day attacks that do not match existing patterns. Furthermore, attackers frequently adapt their methods to bypass signature rules, making these systems increasingly reactive rather than proactive[7][10].

AI/ML-based intrusion detection systems address these limitations by leveraging machine learning models trained on network traffic features to classify behaviors as benign or malicious. Instead of relying solely on predefined rules, these systems learn statistical representations and behavioral relationships within the data. This allows them to detect previously unseen threats, adapt to evolving attack strategies, and refine their accuracy over time. Although AI/ML-based IDS may require higher computational resources—especially during the training phase—they provide significantly enhanced flexibility and detection capabilities, particularly in dynamic and complex network environments[8][10].

The distinction between signature-based IDS and AI/ML-based IDS illustrates a fundamental shift in cybersecurity—from static, rule-driven detection toward intelligent, adaptive systems. This comparison provides a strong foundation for the approach adopted in this project, where Snort is augmented with a Random Forest classifier to overcome its inherent limitations and enable more effective intrusion detection in modern networks.

1.6.3. Snort Architecture and Limitations

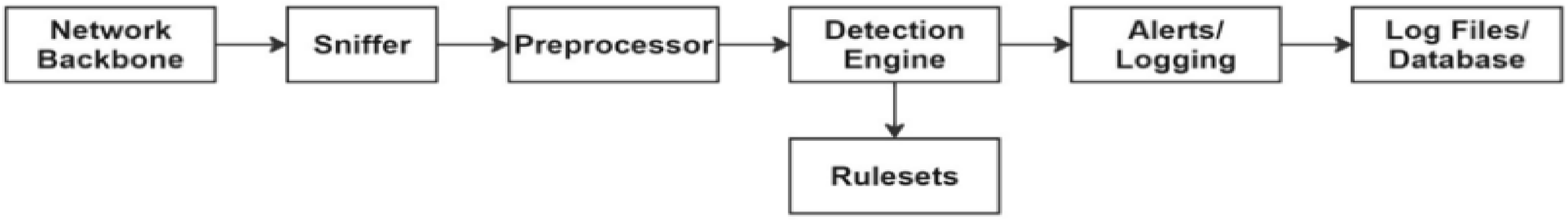


Fig 1.8: Snort Signature Architecture[12].

Snort is one of the most widely deployed open-source Intrusion Detection Systems (IDS), relying primarily on signature-based detection. As illustrated in *Figure 1.8*, the Snort architecture follows a structured pipeline consisting of several components: the **packet sniffer**, which captures raw network traffic; the **preprocessor**, which normalizes packets and removes ambiguities; and the **detection engine**, which compares traffic against a predefined set of rules. When a match is found, Snort generates alerts and stores relevant logs for further analysis[11][12].

This modular architecture provides Snort with several strengths, such as real-time packet inspection, efficient rule matching, and extensive community-maintained rule sets. Its lightweight nature also makes it suitable for deployment on resource-constrained devices, including Raspberry Pi platforms.

Despite these advantages, Snort suffers from several inherent limitations. First, its reliance on **static signatures** restricts its ability to detect zero-day attacks or previously unseen variants. Second, Snort often struggles with **high false-positive rates** when rules are too general or when network behavior changes over time. Additionally, maintaining large and constantly updated rule sets becomes labor-intensive as new threats emerge. These limitations underscore the need for integrating **machine learning (ML)** techniques to enhance Snort's detection capabilities, improve adaptability, and reduce the analyst's manual workload.

The integration approach explored in this project—using a **Random Forest–based risk scoring and classification model**—addresses these shortcomings by analyzing behavioral patterns rather than solely matching predefined signatures.

1.6.4. ML-Enhanced IDS Framework

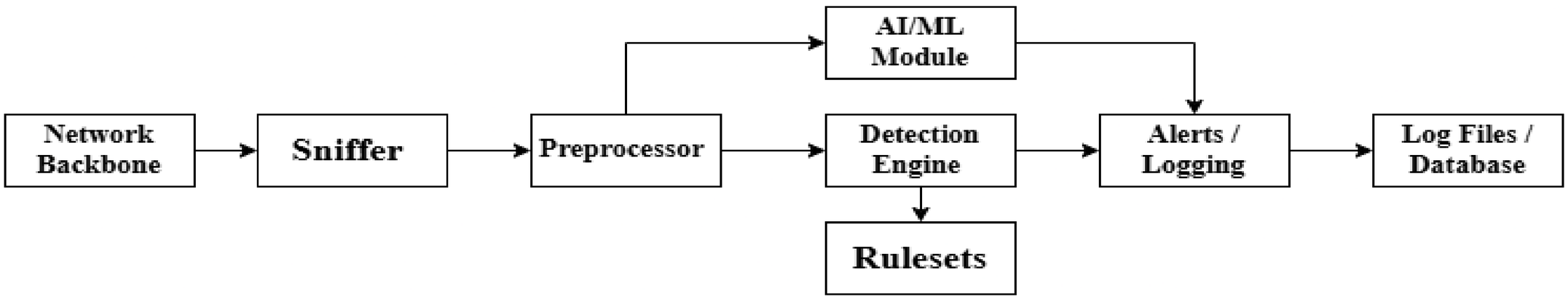


Fig 1.9:Snort with ML-Enhanced Architecture

The integration of Artificial Intelligence (AI) and Machine Learning (ML) into Intrusion Detection Systems represents a significant advancement beyond traditional signature-based detection. The enhanced framework combines Snort's packet inspection capabilities with an intelligent ML processing layer designed to identify complex and previously unseen attack patterns. As shown in the architecture above, the AI/ML module operates alongside Snort's detection engine, receiving preprocessed traffic features and contributing adaptive classification decisions that support more accurate alert generation.

In this framework, Snort continues to perform network packet capture, preprocessing, and rule-based matching. However, instead of relying solely on static signatures, selected features from Snort logs and metadata are forwarded to the ML component for behavioral analysis. The ML engine processes these features, identifies hidden patterns, and determines whether the activity resembles normal or anomalous behavior[3][7][8][10].

Table 1.5:Performance comparison of machine-learning classifiers for intrusion detection[8].

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Logistic Regression	**89.20**	**87.50**	**86.80**	**87.10**
Decision Tree	**91.80**	**90.30**	**89.90**	**90.10**
SVM	**93.50**	**92.10**	**91.80**	**92.00**
Random Forest	**97.20**	**96.80**	**96.50**	**96.70**

The results in *Table 1.4* clearly indicate that the Random Forest classifier outperforms the other evaluated models, namely Logistic Regression, Decision Tree, and SVM, across all key evaluation metrics, including accuracy, precision, recall, and F1-score. Owing to this consistent superiority, Random Forest is selected as the primary classification model in this work, as its ensemble-based decision mechanism enables more robust discrimination between benign and malicious traffic, reduces misclassification rates, and provides a stable foundation for building an effective intrusion detection system[8][10].

Based on these Result, Random Forest was selected for this project as the core machine-learning model to enhance Snort. Its ensemble-based architecture enables it to learn complex decision boundaries and identify subtle deviations in network behavior that signature-based methods cannot detect[8].

In our system, the Random Forest module processes pre-extracted features from Snort alert logs, learns attack signatures from historical datasets, and classifies each event into a risk category (Normal, Low, Medium, High, or Critical). This integration provides three key advantages:

i. **Improved accuracy** in detecting new and modified attacks beyond Snort's rule-based limitations.
ii. **Reduced false positives**, since Random Forest evaluates multiple decision trees before generating an output.
iii. **Adaptive risk scoring** , allowing the IDS to prioritize alerts and support faster analyst response.

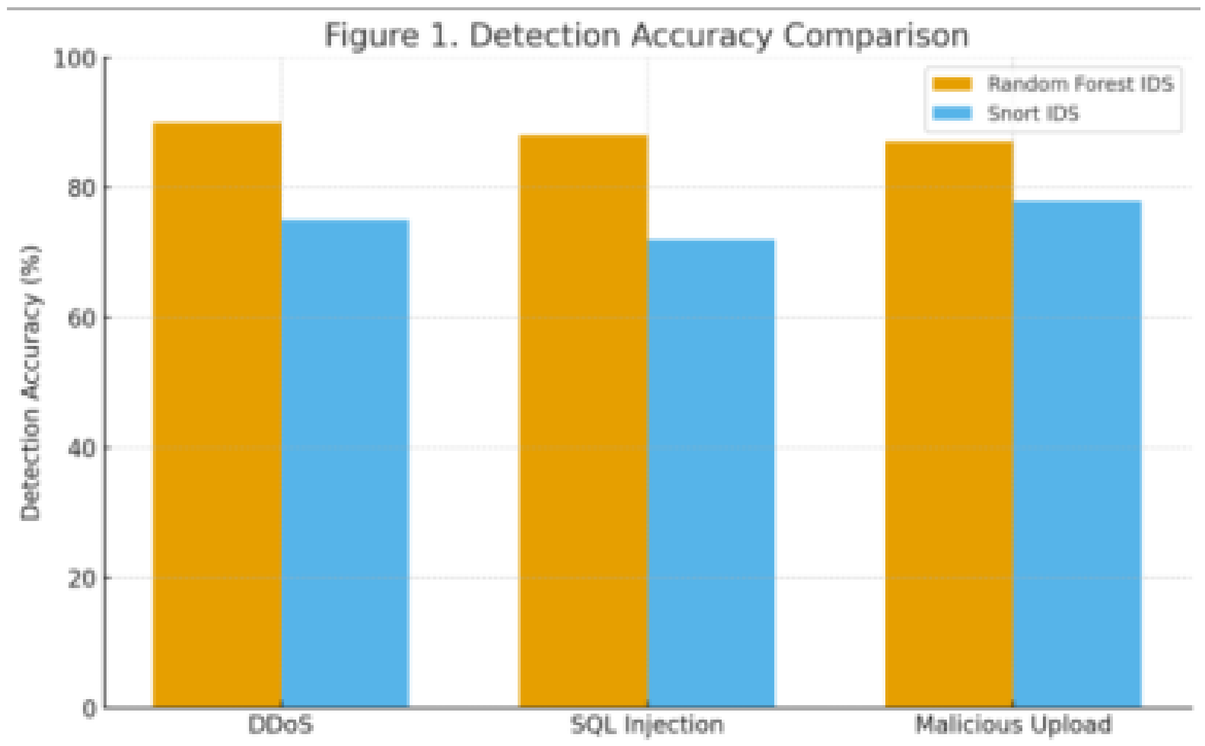


Fig 1.10:Random Forest Detection Accuracy Comparison[13]

When deployed, the trained Random Forest model receives live Snort alerts and analyzes their extracted features to assign a risk category (Normal, Low, Medium, High, or Critical). This automated prioritization step allows the system to highlight high-severity attacks immediately, reducing analyst workload and ensuring that critical threats are addressed before low-impact events.

Experimental comparisons further show that the ML-enhanced IDS achieves noticeably higher detection accuracy than traditional Snort across common attack types such as DDoS, SQL injection, and malicious upload attempts. By learning from real attack datasets and identifying complex patterns beyond predefined signatures, the Random Forest model improves the system's ability to detect evolving threats while reducing missed attacks[8][10][13].

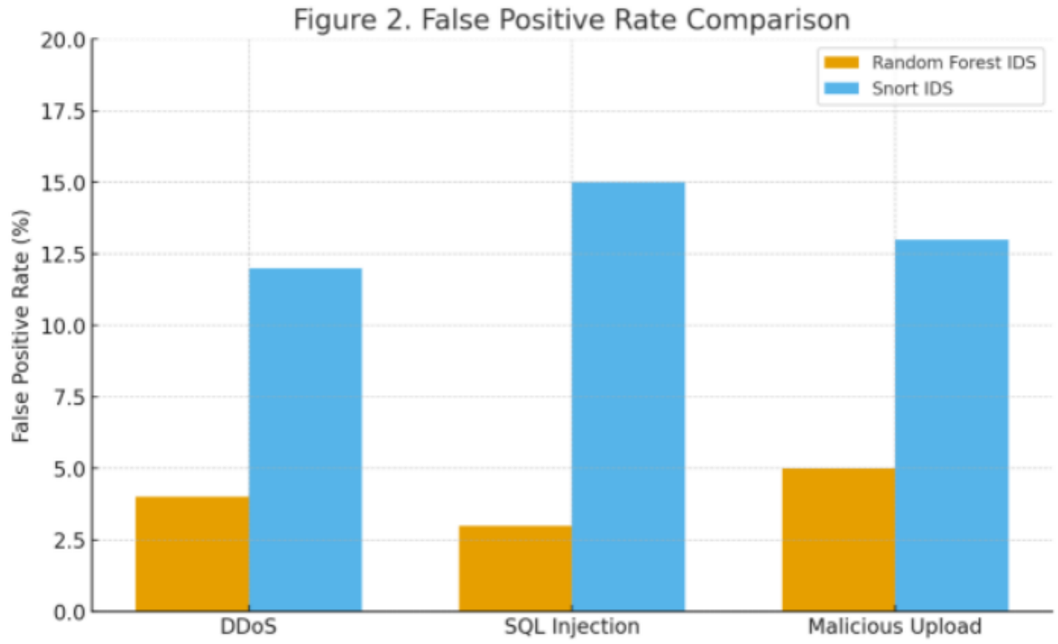


Fig 1.11:False Positive Rate Comparison for Random Forest and Snort IDS[13]

In addition to enhancing detection accuracy, the integration of the Random Forest model significantly reduces the number of false positives—one of the most persistent limitations of traditional signature-based IDS such as Snort. By learning the statistical patterns of both normal and malicious traffic, the ML component is able to more reliably distinguish benign variations from true attack behavior, thereby minimizing unnecessary alerts and reducing analyst fatigue.

Performance evaluations also indicate that the ML-enhanced system maintains manageable computational overhead. Despite the additional processing required for feature extraction and classification, both CPU utilization and processing latency remain within acceptable operational limits. This demonstrates that the proposed solution improves detection quality without imposing excessive resource consumption, making it suitable for real-time deployment[13].

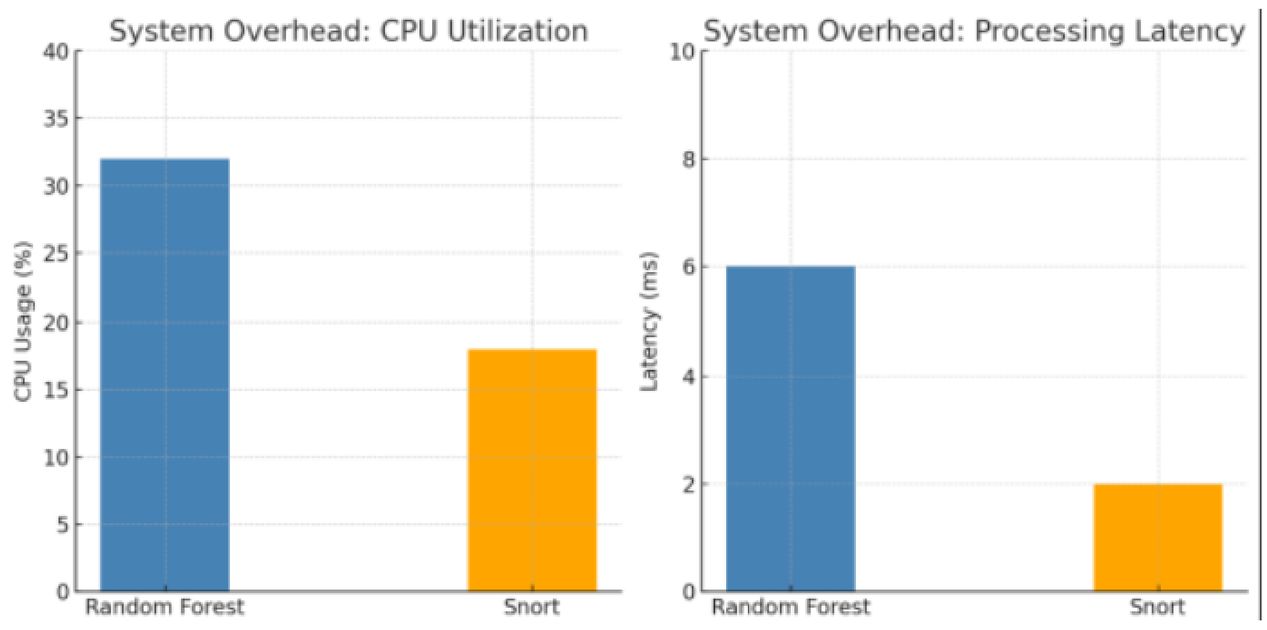


Fig 1.12:CPU Utilization and Processing Latency for Random Forest vs. Snort IDS[13].

Overall, the ML-enhanced IDS framework provides a more adaptive and intelligent defense mechanism capable of addressing modern and evolving cyber threats. By combining rule-based inspection with machine learning–driven analysis, the system delivers improved threat detection, better risk prioritization, and a stronger security posture compared to conventional IDS deployments[13].

1.6.5. Related Work Comparison

Understanding prior research in intrusion detection is essential to contextualizing the strengths and limitations of existing IDS methodologies. Recent studies have explored hybrid Snort–AI architectures, standalone machine-learning-based IDS models, and performance evaluations on widely used datasets. *Table 1.4* summarizes the most relevant studies, highlighting their techniques, datasets, ML models, performance results, and limitations.

Table 1.6:Comparison of Related Intrusion Detection Studies

Title(Year)	Technique	Dataset	ML Model Used	Results (Accuracy / FPR)	Limitations
Enhancing Intrusion Detection and Cloud Security by Integrating Snort with Advanced AI Techniques for Improved Accuracy and Threat Mitigation (2025) [3].	Hybrid Snort + AI (feature selection + ML filter on Snort alerts)	UNSW-NB15	Random Forest, Isolation Forest, SVM	94.6% Accuracy / 2.7% FPR	High CPU/memory overhead (42-50% increase); simulation-only testing
Enhanced Intrusion Detection Systems Performance with UNSW-NB15 Data Analysis (2024)[14].	ML-based IDS with correlation-based feature selection and comparison of multiple classifiers	UNSW-NB15	LR, SVM, DT, RF, XGBoost	Best accuracy: 99.45%, FAR: 1.94%	Only offline testing; high danger; no Snort integration; no deployment of edge devices
Advanced Random Forest for Real-Time IDS (2024)[15]	Optimized RF for real-time intrusion detection	UNSWNB15, CIC-IDS2017	Random Forest	97.3% UNSW-NB15, 98.9% CICIDS2017	Limited testing on real-world traffic.
Comparative Study of Random Forest (ML) and Snort (Signature) IDS: Detection Performance (Accuracy & False Alarms) and Operational Overhead in Public Cloud Deployments (2025)[13]	Direct comparison of RF-IDS vs Snort signature IDS	Cloud/ AWS custom dataset	Random Forest	RF >90% accuracy, FPR <5%; Snort ≈80% accuracy, FPR 12–15%	Limited attack coverage and higher processing cost on ML

1.6.6. Research Gap

The review of existing literature reveals several gaps that this project aims to address:

- **Most machine-learning-based IDS studies are evaluated offline**, using datasets only, without integrating ML models into real-time IDS engines such as Snort. This means their performance is not validated under realistic network traffic conditions.

- **Limited research explores hybrid Snort–ML architectures**, and existing studies do not thoroughly examine how signature-based detection and ML classification can operate together in a unified detection pipeline.
- **Deployment feasibility on lightweight hardware (e.g., Raspberry Pi) is rarely evaluated**, making many solutions impractical for real-world or resource-constrained environments.
- **Few studies introduce risk scoring or alert prioritization**, focusing mainly on binary classification (attack/normal) without assigning severity levels that support faster and more informed incident response.
- **No existing work provides a complete end-to-end implementation** combining Snort packet inspection, ML classification, and automated risk scoring into an integrated system suitable for operational use.

This project addresses these gaps by developing a unified Snort–Random Forest framework that improves detection accuracy, reduces false positives, supports real-time risk scoring, and is deployable on low-cost hardware.

1.7. Significance

The significance of this project lies in its ability to address critical limitations in traditional intrusion detection systems and to provide a more adaptive, accurate, and operationally effective security solution. As cyber threats continue to evolve in complexity and frequency, enhancing detection mechanisms with intelligent machine-learning capabilities becomes essential for modern network defense. The proposed hybrid Snort–Random Forest system offers several key benefits that contribute to improving cybersecurity readiness and operational efficiency.

First, the system significantly enhances detection accuracy by combining rule-based inspection with machine-learning-based behavioral analysis. This hybrid approach allows for more reliable identification of both known and emerging attack types, addressing Snort's inherent limitations in detecting zero-day or modified threats. **Second**, the integration of a machine-learning model reduces false-positive alerts, which are a major source of analyst fatigue in traditional IDS platforms. By minimizing unnecessary alerts, the system supports more efficient incident triage and reduces response delays.

Third, the inclusion of a real-time risk-scoring mechanism provides actionable insight into the severity of detected incidents. Assigning each alert a risk level (Normal, Low, Medium, High, Critical) enables security teams to prioritize high-impact threats and allocate resources more effectively. **Fourth**, the system is designed to be lightweight enough for deployment on devices such as Raspberry Pi, making it suitable for small organizations, educational environments, and distributed monitoring scenarios where cost and hardware constraints are critical factors.

Finally, the project contributes to the academic and research community by providing a practical, deployable framework that demonstrates how machine learning can be effectively integrated into existing IDS infrastructures. This work can serve as a foundation for future advancements, such as incorporating deep learning models, adaptive feature selection, or automated response mechanisms.

Chapter 2.

SYSTEM DESIGN ALTERNATIVES

2.1. Design Requirements

This section identifies the system needs and translates them into measurable design requirements necessary for developing the hybrid Snort–AI intrusion detection system. The design requirements ensure that the system achieves high detection accuracy, maintains operational efficiency on resource-constrained hardware, and supports risk-based alerting.

2.1.1. System Needs

Based on the problem definition and operational goals of the proposed IDS, the following system needs were identified:

1) A mechanism to detect both known and previously unseen cyberattacks.
2) A reduction in the high false-positive rates typically associated with signature-based IDS.
3) An intelligent classification component capable of assigning severity levels to detected incidents.
4) Compatibility with resource-constrained hardware platforms such as Raspberry Pi.
5) An efficient alerting and logging mechanism to support incident response[16][17].

2.1.2. Derived Design Requirements

Based on the system needs, the following design requirements are established:

1) The IDS must integrate both signature-based detection (Snort) and machine-learning classification.
2) The system must achieve high detection accuracy while reducing false positives compared to Snort alone.
3) The ML module must provide five-level risk scoring (Normal, Low, Medium, High, Critical).
4) The solution must operate within the processing and memory limits of Raspberry Pi without exceeding acceptable overhead.
5) The system must maintain structured logging of alerts, classifications, and risk levels for post-analysis[16].

2.1.3. Needs–Requirements Mapping Table

Table 2.1:Mapping Between System Needs and Design Requirements

Needs	Design requirements
Able to monitor network traffic	• Has Snort IDS installed on Raspberry Pi • Captures real-time network packets
Able to detect and classify threats	• Has Random Forest Machine Learning model • Classifies alerts into Normal, Low, Medium, High, Critical
Able to prioritize and assign risk	• Calculates risk score for each alert • Sorts alerts based on severity to focus on critical threats
Able to alert and notify users	• Has dashboard or interface to display alerts • Can send notifications for high-risk threats
Able to store and process data	• Has storage for logs and network data • Has processing unit to run ML model and Snort efficiently

2.2. Design Alternatives

This section presents the feasible design alternatives considered for implementing an effective intrusion detection system. Each alternative differs in terms of architectural complexity, processing capabilities, machine learning integration, cost, and deployment feasibility. A comparative understanding of these alternatives is essential before selecting the most appropriate design approach for the project.
All alternatives are evaluated based on technical performance, scalability, hardware requirements, real-time processing capability, and overall practicality for deployment in constrained environments.

The following subsections describe the three proposed alternatives, including their conceptual design, architectural workflow, advantages, and limitations[18][19].

2.2.1. Summary of Feasible Design Approaches

Before selecting the final system design, three approaches were evaluated:

Alternative 1 **– Traditional IDS Snort Signature-Based IDS**
Relies solely on Snort's signature engine without machine learning components.

Alternative 2 **– IDS Snort + Deep Learning IDS (NVIDIA Jetson Orin)**
Enhances Snort by integrating a deep neural network trained using CICIDS2017.

Alternative 3 **–IDS Snort + Random Forest + Risk Scoring**
A combined approach that leverages Snort for signature detection and Random Forest for intelligent attack classification and risk scoring.

2.2.2. Design Alternative 1: Traditional IDS Snort Signature-Based IDS

Description

This alternative uses traditional Snort as the primary detection engine. Snort analyzes incoming packets, applies its predefined rule set, and generates alerts whenever a signature match is found. This approach is simple, lightweight, and widely used in many production environments[6].

Flowchart

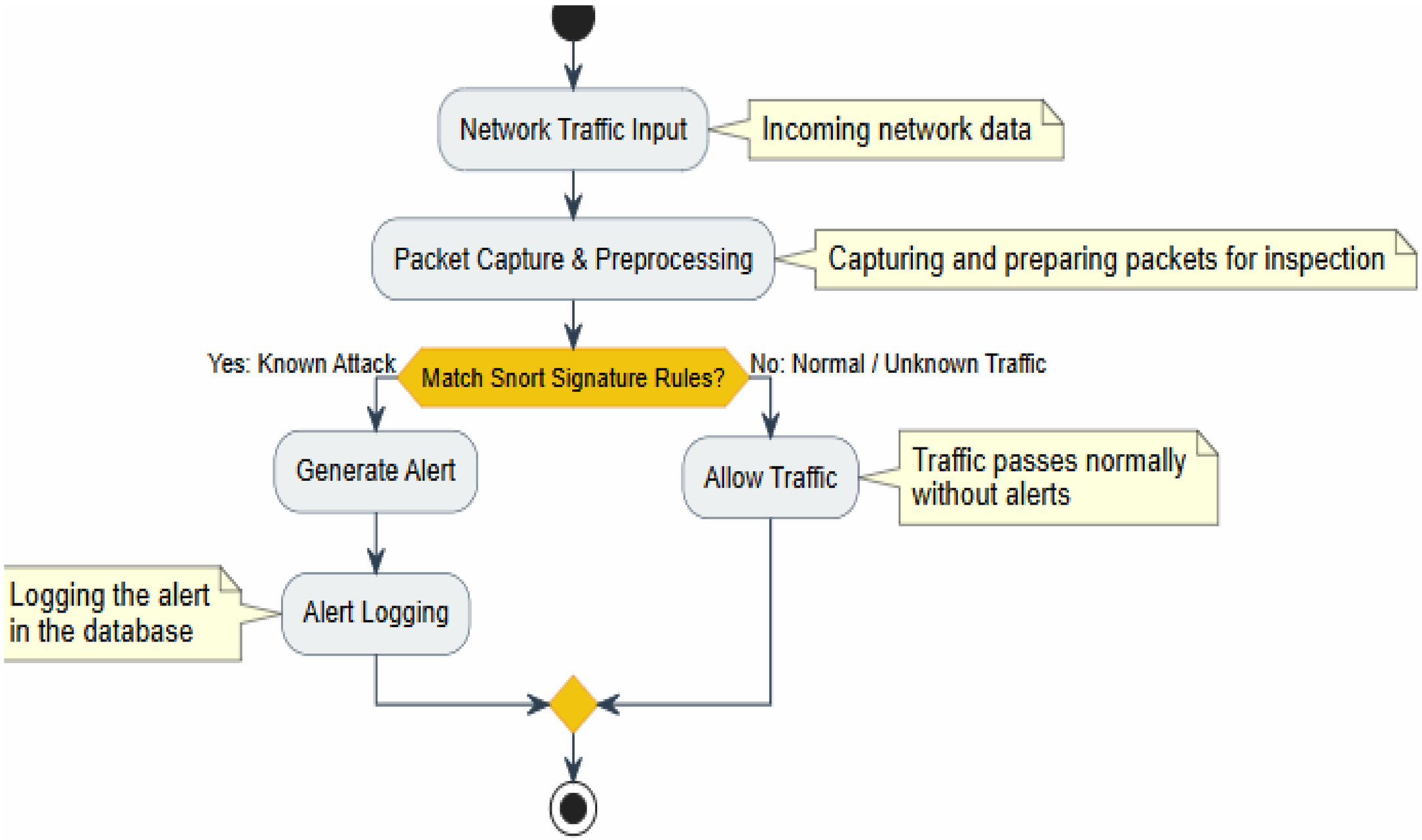


Fig 2.1: Design Alternative 1

Advantages

- Simple to deploy and maintain.
- Low cost and Reliable for known attacks

Limitations

- Cannot detect zero-day or unknown attacks.
- High false-positive rates in dynamic networks.
- No intelligent threat prioritization.

2.2.3. Design Alternative 2: IDS Snort + Deep Learning IDS (NVIDIA Jetson Orin)

Description

This alternative enhances Snort by integrating a deep learning module running on NVIDIA Jetson Orin. Snort performs packet capture and preprocessing, while the neural network (trained using the CICIDS2017 dataset [16][17][18].

Flowchart

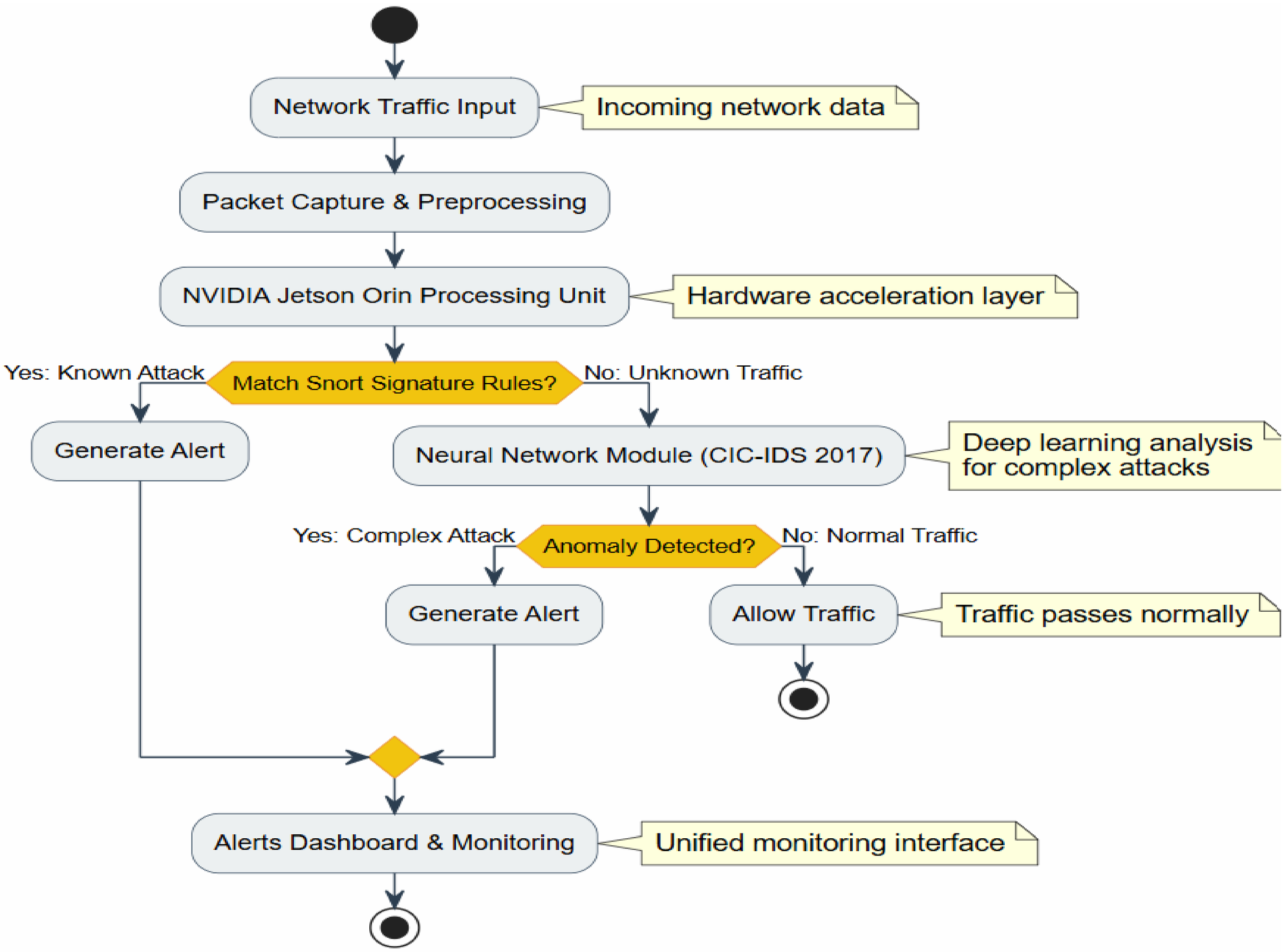


Fig 2.2: Design Alternative 2

Advantages

- Can detect unknown or complex attacks
- Higher accuracy than signatures alone
- Jetson Orin provides strong processing power

Limitations

- Higher cost and energy consumption
- Requires a more complex training process
- Not ideal for low-power environments like Raspberry Pi
- Integration may introduce extra delay

2.2.4. Design Alternative 3: IDS Snort + Random Forest + Risk Scoring

Description

This design integrates Snort with a Random Forest classifier and a risk-scoring module. Snort creates basic alerts, and the Random Forest model classifies attacks based on the CICIDS2017 dataset A risk-scoring module prioritizes alerts based on severity[16][19].

Flowchart

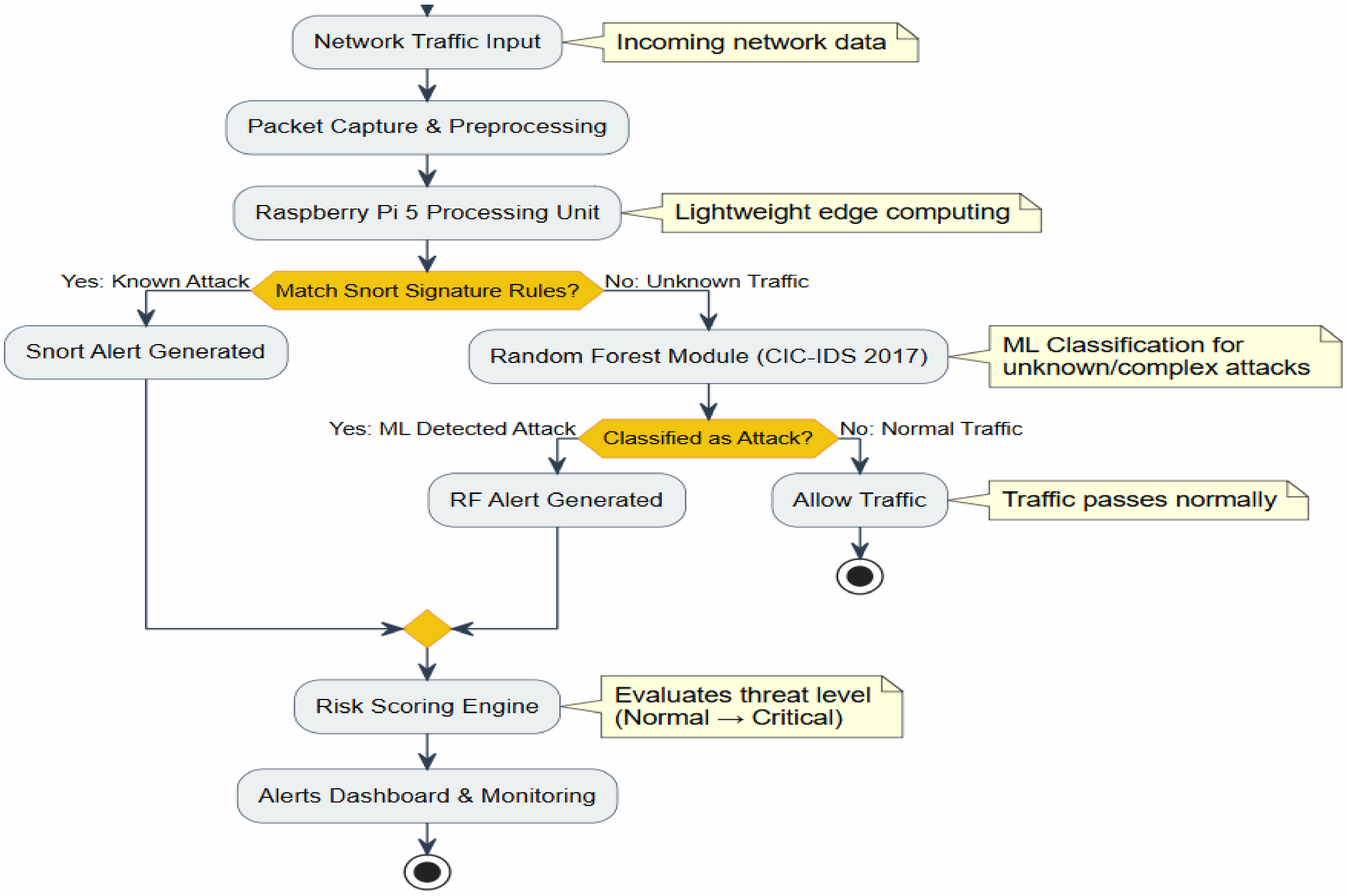


Fig 2.3: Design Alternative 3

Advantages

- Detects known and unknown attacks
- Low false-positive rate
- Real-time severity scoring
- Efficient enough for embedded hardware (Raspberry Pi)

Limitations

- Requires periodic ML model updates
- More complex than a traditional IDS

2.3. Design Alternatives Evaluation Criteria

This section outlines the evaluation method used to compare the three proposed design alternatives. The goal is to select the most appropriate system architecture based on functional capabilities, performance, and practical feasibility. The evaluation is based on a scoring system commonly used in engineering design reviews, where each requirement or criterion is assigned a score To evaluate the three design alternatives objectively, several factors were identified and assigned weights according to their importance to the project. These factors reflect both technical and operational requirements essential for building a functional and scalable intrusion detection system.

This scoring model ensures an objective comparison among the alternatives and reflects how well each design satisfies the necessary intrusion detection and operational features.

2.3.1. Evaluation Factors

The evaluation factors consist of two categories:

1) Design Requirements

These represent essential capabilities the system must provide, such as detecting attacks, classifying threats, prioritizing alerts, and operating effectively on embedded hardware.

2) Evaluation Criteria

These include system-level performance characteristics such as detection accuracy, cost efficiency, computational efficiency, and real-time capability.

Each alternative is evaluated against all requirements and criteria using the scoring system defined above.

2.3.2. Decision Matrix

The following decision matrix summarizes how each of the three design alternatives performs when evaluated against all requirements and criteria. Scores reflect the degree to which each alternative meets the project's objectives.

The decision matrix provides a structured method for scoring and comparing the three design alternatives using the identified evaluation factors and associated weights. Each alternative is evaluated using a grading scale:

- **0 = does not fit**
- **1 = somehow fit**
- **2 = somehow fit**

The weighted score for each criterion is calculated by multiplying the weight ($\boldsymbol{v}$) by the assigned grade ($\boldsymbol{w}$). The total score for each design alternative is used to determine the most suitable architecture for implementation.

Table 2.2:Decision Matrix for the Three Design Alternatives

Design Requirements / Criteria			Design Alternative #1		Design Alternative #2		Design Alternative #3	
	Name	w	v	$w \times v$	v	$w \times v$	v	$w \times v$
Design requirements	Ability to detect attacks	0.20	1	0.20	2	0.40	2	0.40
	Ability to classify threats	0.15	0	0.00	2	0.30	2	0.30
	Ability to prioritize alerts	0.15	0	0.00	0	0.00	2	0.30
	Hardware compatibility	0.10	2	0.20	0	0.00	2	0.20
Criteria	Detection accuracy	0.15	1	0.15	2	0.30	2	0.30
	Cost efficiency	0.10	2	0.20	0	0.00	1	0.10
	Computational efficiency	0.10	2	0.20	0	0.00	2	0.20
	Real-time capability	0.05	2	0.10	1	0.05	2	0.10
Total Points		**1**	**10**	**1.05**	**7**	**1.05**	**15**	**1.9**

2.3.3. Selected Design Alternative

Based on the results of the decision matrix, **Design Alternative 3 — Snort + Random Forest + Risk Scoring achieves** the highest overall score and demonstrates the strongest alignment with the system requirements and evaluation criteria. The superior performance of this alternative is attributed to its ability to integrate the strengths of both signature-based and machine-learning–based detection approaches, resulting in a more adaptive and intelligent intrusion detection framework.

Unlike the traditional Snort-only solution, which relies solely on predefined signatures, the proposed hybrid design leverages Random Forest classification to detect both known and previously unseen attack patterns. This significantly enhances the system's detection accuracy, particularly against zero-day and evolving threats, which represent a major challenge for conventional IDS platforms.

Considering all these strengths—higher accuracy, reduced false positives, intelligent alert prioritization—the Snort + Random Forest + Risk Scoring architecture clearly represents the most optimal and balanced solution for the project's goals. ***Design Alternative 3*** is so chosen.

2.4. System Requirements

The system requirements define the essential capabilities and constraints necessary for implementing the proposed Intrusion Detection System. These requirements ensure that the system meets its functional objectives, performs efficiently on embedded hardware, and supports reliable real-time threat detection and risk analysis.

2.4.1. Functional Requirements

The functional requirements define the essential capabilities that the proposed IDS system must provide to ensure effective detection, classification, and prioritization of network threats. These requirements reflect the core operational behaviors expected from the system during real-time monitoring, analysis, and response.

Table 2.3:Functional Requirements List[6][18]

FR.No	Functional Requirement	Description
FR1	Network Traffic Monitoring	The system shall continuously capture and inspect network packets in real time.
FR2	Signature-Based Detection	The system shall use Snort to detect attacks based on predefined rules and signatures.
FR3	Machine-Learning-Based Classification	The system shall classify alerts using a Random Forest model to identify normal and malicious behavior.
FR4	Risk Scoring	The system shall assign a severity level (Normal, Low, Medium, High, Critical) to each detected incident.
FR5	Alert Prioritization	The system shall prioritize alerts based on risk levels to support efficient incident response.
FR6	Logging and Storage	The system shall record all alerts, classifications, and risk scores for analysis and documentation.
FR7	Dashboard Visualization	The system shall display the system status, alert categories, and risk levels through a real-time dashboard.

2.4.2. Hardware Requirements

The hardware requirements specify the physical components necessary to deploy the IDS system. These components must support real-time packet capture, machine-learning inference, logging, and dashboard visualization while operating efficiently on a resource-constrained platform.

Table 2.4:Hardware Requirements

Hardware Component	Specification / Description
Raspberry Pi (Model 4 or higher)	Acts as the main device that runs Snort, the machine-learning scripts, and the system dashboard.
MicroSD Card (256 GB)	Used for OS installation and Stores Snort files, log files, and machine-learning data.
Network Interface (Built-in or external)	Allows the system to capture network traffic and monitor data in real time.
Power Supply (5V / 3A)	Provides stable and continuous power to the Raspberry Pi during operation.

2.4.3. Software Requirements

The software requirements identify the tools, libraries, and frameworks needed to implement the IDS system. These components form the software stack that supports packet inspection, feature extraction, machine learning, and visualization.

Table 2.5:— Software Components and Descriptions

Software Component	Description
Snort IDS	Detects attacks by checking network traffic in real time using predefined rules.
Python 3	Programming language used for data processing and ML integration.
Scikit-Learn	Provides the Random Forest implementation used for classification.
Ubuntu OS	Runs all system software, including Snort, Python, and the dashboard.
Flask / Dashboard Framework	Displays alerts, risk levels, and system status.

2.4.4. Non-functional Requirements

The non-functional requirements outline the efficiency and effectiveness requirements that the system must fulfill in order to guarantee dependability, usability, and operational efficacy. These requirements do not describe system behavior directly but define constraints and performance expectations.

- **Performance:** The system must process incoming packets and generate classifications with minimal delay to support real-time monitoring.

- **Scalability:** The system must be able to analyze increasing traffic volumes without significant degradation of performance.

- **Reliability:** The system must operate continuously and maintain stable detection accuracy over prolonged periods.

- **Usability:** The dashboard interface must provide clear, intuitive visualizations that assist analysts in quickly understanding system status.

- **Maintainability:** Snort rules and the machine learning model must be updatable to incorporate new attack patterns and datasets.

- **Security:** The system must ensure secure handling of log files and prevent unauthorized modification of detection configurations

2.5. .Realistic Constraints

Every engineering project operates under a set of realistic constraints that influence design decisions, limit available options, and shape the final system architecture. For this Intrusion Detection System (IDS) project, several factors—including hardware cost, environmental considerations, ethical responsibilities, social impact, regulatory compliance, physical restrictions, and sustainability—played a significant role in directing the design choices. This section outlines the main challenges and constraints encountered during the development of the proposed Snort + Random Forest + Risk Scoring system.

2.5.1. Economic Constraints

Economic limitations directly affected the selection of hardware and software components for this project. The system needed to operate on an affordable platform suitable for students, small organizations, and budget-limited environments. Raspberry Pi was selected due to its low cost, availability, and compatibility with Snort and Python-based ML models. Additional cost considerations included storage media, power supply, and cooling components.

Moreover, the use of open-source tools such as Snort, Python, Scikit-Learn, and Flask significantly reduced the overall software cost. The design ensures that, if mass-produced, the system would remain economically feasible due to its reliance on inexpensive materials and low energy consumption. The economic constraint encouraged a lightweight implementation that avoids high-end hardware such as GPUs or deep learning accelerators[16][19].

2.5.2. Environmental Constraints

The environmental impact of the system was considered in terms of electronic waste, materials used in hardware components, and long-term energy consumption. Raspberry Pi and related components produce minimal heat and have relatively low power requirements, making them suitable for environmentally conscious deployments.

Environmental conditions in operational environments—such as temperature, dust, and electrical noise—were also taken into account, as they may influence system stability. The project must avoid excessive energy usage, rely on environmentally safe materials, and align with local environmental regulations related to electronic equipment disposal.

Overall, the system's low power profile contributes positively by reducing energy demand compared to traditional servers or GPU-based IDS systems[18][20].

2.5.3. Ethical Constraints

The system involves monitoring network activity, which introduces ethical considerations related to privacy, data handling, and potential misuse. The design must comply with the IEEE Code of Ethics and ensure responsible use of collected data. Unauthorized packet inspection or surveillance can lead to ethical violations; therefore, deployment must occur only in environments where monitoring is legally permitted and users are informed.

Additionally, the analysis must avoid storing unnecessary personal data and follow best practices for secure data handling. Ethical constraints encouraged a transparent design that prioritizes user trust and responsible cybersecurity practices.

2.5.4. Health and Safety Constraints

The hardware components operate at low voltage (5V), reducing electrical safety risks. However, continuous operation requires adequate cooling to prevent overheating. Proper ventilation, heat sinks, or small fans ensure safe temperature levels. Users must handle electronic components cautiously during installation to avoid short circuits or physical damage.

No harmful emissions or dangerous materials are involved in normal operation, and the design complies with relevant safety guidelines for low-power embedded systems. Clear usage instructions must accompany the system to ensure safe installation and operation.

2.5.5. Social Constraints

The deployment of intrusion detection systems affects users by monitoring their network activities. Social constraints include concerns about trust, transparency, and the balance between security and privacy. Proper communication with users and stakeholders is necessary to ensure acceptance of the technology.

The system should not disrupt daily activities or create fear of surveillance. Instead, it should provide reassurance by enhancing cybersecurity posture. Social considerations also involve the accessibility of the system—ensuring it is easy to use, understandable, and deployable in academic, small business, or home environments.

2.5.6. Political Constraints

Political and legal factors restrict how intrusion detection tools can be used. Network monitoring must comply with national cybersecurity laws, data-protection regulations, and telecommunications policies. Unauthorized monitoring may violate legal frameworks governing privacy, encryption, and digital rights.

In Saudi Arabia, the deployment of network monitoring tools such as IDS must comply with regulations set by the National Cybersecurity Authority (NCA) and the Anti-Cybercrime Law. These laws restrict unauthorized monitoring of network traffic and require systems to be used only within authorized private networks. Therefore, the proposed IDS must operate strictly within controlled environments, ensuring that packet inspection does not violate privacy or regulatory guidelines.

2.5.7. Physical Constraints

The system is physically constrained by the compact size and thermal limitations of Raspberry Pi hardware. Weight and dimensions limit the use of large cooling systems or additional hardware modules. The device must be placed in a safe, stable environment that supports proper airflow and reliable network connectivity.

Limited onboard processing power and memory also influenced design decisions, requiring a lightweight ML model such as Random Forest rather than resource-intensive deep learning models.

2.5.8. Sustainability Analysis

Lifecycle evaluation considers financial ecological societal consequences overall. This includes material use, cost, energy consumption, long-term usability, and community impact.

- **Economic Sustainability**

Economic sustainability considers prototype cost, potential mass-production savings, and energy efficiency. The system uses low-cost components and open-source software, reducing development and operational expenses. Long-term energy savings compared to full server-based IDS platforms make the design economically viable.

- **Environmental Sustainability**

Environmental sustainability evaluates energy consumption, waste generation, and reliance on scarce resources. Raspberry Pi's low power usage reduces greenhouse gas emissions, and the hardware relies mostly on widely available materials. The system generates minimal electronic waste and supports environmentally conscious deployment.

- **Social Sustainability**

Social sustainability examines how the project affects people's lives, community needs, and employment. The IDS enhances cybersecurity awareness and protection, improves organizational resilience, and can create educational and operational opportunities. It does not replace human labor but assists analysts by reducing alert fatigue and improving response efficiency.

- **Long-Term and Lifecycle Considerations**

Long-term sustainability involves evaluating maintenance needs, upgrade pathways, recyclability, and adaptability to future technologies. The system supports modular updates, allowing hardware or ML models to be upgraded without replacing the entire system. Lifecycle analysis ensures the system remains efficient, safe, and environmentally responsible through deployment, operation, and end-of-life stages.

2.6. Standards to be Used

This section lists the technical standards relevant to the design and operation of the proposed Intrusion Detection System (IDS). These standards ensure compatibility, security, and correct functioning of the hardware, software, and communication layers used in the project.

Ensuring compliance with recognized technical standards is essential for maintaining the reliability and interoperability of the proposed system. Because the IDS integrates multiple subsystems—including network communication modules, Linux-based software services, machine-learning components, and open-source security tools—it must operate within a consistent and well-established standards framework. These standards define how data is captured, processed, secured, and evaluated across the entire pipeline, enabling stable performance and reducing compatibility issues during deployment. They also provide guidelines for encryption, communication protocols, and software execution, which strengthens overall system integrity and ensures that the IDS behaves predictably under different operating conditions. The main standards applied in this project are presented in *Table 2.6*.

Table 2.6:Standards to Be Used

Category	Standard	Application in the Project
Network Communication Standards	**IEEE 802.3 (Ethernet)**	Ensures proper operation of packet capture over wired network interfaces on the Raspberry Pi.
	IEEE 802.11 (Wi-Fi)	Ensures compatibility with wireless traffic when Snort monitors Wi-Fi networks.
Security & Encryption Standards	**AES (FIPS-197)**	Provides secure encryption for stored log files and alerts when needed.
	TLS/SSL (IETF RFC 5246 / RFC 8446)	Ensures secure communication if the dashboard or alerts are accessed remotely.
Software & Operating System Standards	**POSIX (IEEE 1003)**	Ensures system commands, Python scripts, and Snort run correctly on Linux-based Raspberry Pi OS.
	GNU/Linux Standards	Required for using open-source Linux components such as Snort and system utilities.
Machine Learning Guidelines	**NIST IR 8269**	Provides recommended practices for reliability and evaluation of machine learning models such as Random Forest.
Open-Source Compliance	**GPL License (Snort)**	Ensures legal use of Snort IDS under its open-source license.
	Apache License 2.0 (Scikit-Learn)	Governs legal use of ML libraries integrated with the project.

2.7. Cost Analysis

Cost analysis evaluates the financial feasibility of implementing the proposed hybrid Snort–Machine Learning intrusion detection system. Since the design relies on open-source technologies and cost-efficient hardware such as the Raspberry Pi, the overall financial requirements remain low, making the system accessible for educational, research, and small-scale deployment environments.

To provide a comprehensive understanding of the financial requirements of the proposed system, the remainder of this section is divided into three detailed parts. The first part presents the hardware cost breakdown, outlining the essential physical components needed to deploy the IDS and their associated price ranges. The second part discusses the software cost breakdown, emphasizing the reliance on open-source tools that significantly reduce overall expenses while still delivering advanced functionality. The third part consolidates these findings into a total estimated cost, offering a clear summary of the project's affordability and demonstrating its suitability for academic, research, and small-scale operational use.

Overall, this structured cost analysis ensures transparency in budgeting and helps evaluate the practicality of implementing the system in real-world environments. By presenting the costs in a clear and organized manner, the project demonstrates both technical feasibility and financial efficiency.

2.7.1 Hardware Cost Breakdown

The hardware components used in this project were selected based on affordability, availability, and compatibility with lightweight IDS and ML workloads. *Table 2.7* summarizes the estimated cost for each component.

Table 2.7:Hardware Cost Analysis[16][17]

Hardware Component	Estimated Cost (SR)
Raspberry Pi 4 (4GB RAM)	**250–320 SR**
MicroSD Card (32–64 GB)	**20–40 SR**
Raspberry Pi Protective Case	**15–30 SR**
Cooling Fan / Heat Sink	**10–25 SR**
Network Interface (Built-in or External USB NIC)	**0–35 SR**
Power Supply (5V / 3A)	**20–30 SR**
External USB Storage (Optional)	**25–60 SR**

Estimated Hardware Total Cost: 340-540 SR

2.7.2 Software Cost Breakdown

All software tools used in this project are open-source and freely available. This significantly reduces deployment costs and increases accessibility.

Table 2.8:Software Cost Analysis[19]

Software Component	Cost
Snort IDS	**0 SR**
Python 3	**0 SR**
Scikit-Learn	**0 SR**
Linux OS (Raspberry Pi OS)	**0 SR**
Flask / Dashboard Framework	**0 SR**

Estimated Software Total Cost: 0 SR (*All components are open-source*)

2.7.3 Total Estimated Cost

The following table summarizes the total estimated cost required to implement the proposed IDS system, including all essential hardware components needed for deployment.

Table 2.9:Total System Cost Summary

Category	Total Cost (SAR)
Hardware	**340-540 SR**
Software	**0 SR**
Total Cost	**340-540 SR**

Note:

The total estimated cost includes only the essential hardware components required to deploy the proposed intrusion detection system on the Raspberry Pi platform.

These components cover processing, storage, cooling, and network connectivity needed for continuous operation.

Optional items such as external storage are included only when beneficial for extended log retention. All software tools used in the system, including Snort, Python, Scikit-Learn, and the dashboard framework, are fully open-source and therefore add no additional financial cost.

This makes the overall solution highly economical and suitable for academic and small-scale deployment environments.

Chapter 3.

SYSTEM DESIGN AND IMPLEMENTATION

3.1. Detailed Design Description

3.1.1. Overall System Description

The suggested system is a intrusion detection framework that combines a machine learning-based classification model with a traditional signature-based detection engine. Real-time network traffic monitoring, malicious activity detection, and assigning a risk level to each observed event are all features of the system.

The Snort Intrusion Detection System starts the detection process by using pre-defined rules to examine incoming network packets. Snort creates alarm logs that are sent to the processing stage after it detects suspicious traffic. After that, these alerts are converted into structured data that may be analyzed using machine learning.

A Random Forest model is used to classify network events based on extracted features. The model enhances detection performance by identifying both known and unknown attack patterns. Following classification, a risk scoring mechanism assigns a numerical value to each event, which is then mapped into predefined categories ranging from Normal to Critical[11].

The final output is presented through a dashboard interface that allows users to monitor alerts and prioritize responses based on severity levels.

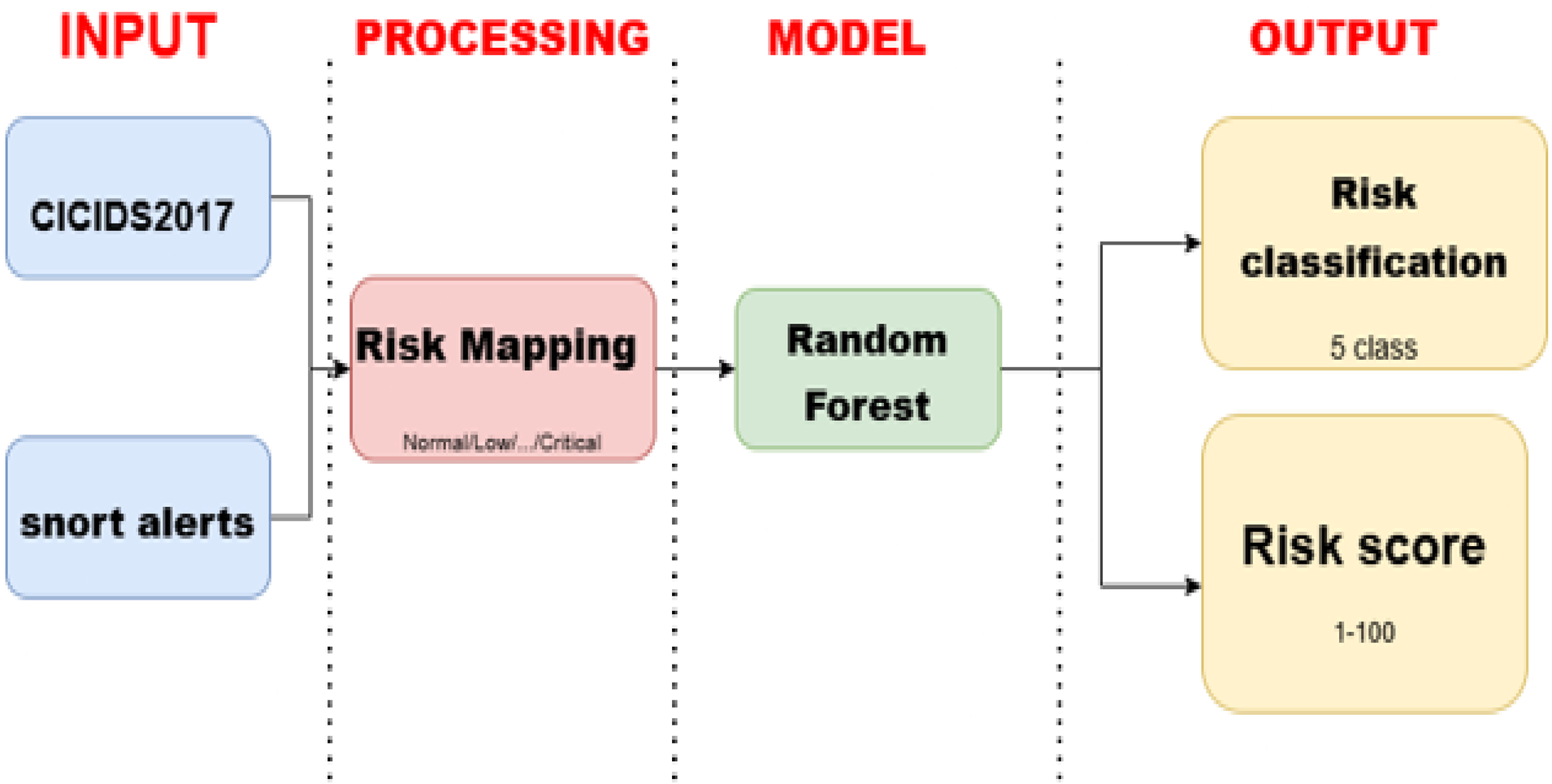


Fig 3.1:System Workflow

3.1.2. System Components Description

The system is composed of several interconnected components, each responsible for a specific function within the detection pipeline.

The Snort module is responsible for capturing and analyzing network traffic using signature-based rules. It generates alerts whenever suspicious activity is detected. The alert processing module transforms raw alert logs into structured data by extracting relevant attributes such as timestamps, protocols, and source/destination information.

The machine learning module utilizes a Random Forest classifier to analyze processed data and predict the likelihood of malicious behavior. This module improves the system's ability to detect the zero day attacks.

The risk scoring module assigns a numerical score to each classified event and categorizes it into predefined severity levels. Finally, the dashboard module visualizes the results, enabling users to quickly assess threats and take appropriate actions.

Table 3.1: System Components

Component	Function
Raspberry Pi 5	IDS hardware device
MicroSD Card	System storage
Ubuntu	Operating system
Network Interface (`wlan0` / `eth0`)	Capture network traffic
Snort	Detect suspicious traffic
`local.rules`	Define attack detection rules
`snort.lua`	Configure Snort settings
Alert Logs	Store intrusion alerts
Alert Processing Module	Convert alerts to structured data
Python	Backend programming environment
Flask	Run web dashboard
scikit-learn Random Forest	Classify attacks
`inference_engine.py`	Predict attack type
Dataset Module	Load training/testing data
`safe_evaluate.py`	Evaluate model accuracy
`scaler.pkl`	Normalize features
`label_encoder.pkl`	Decode prediction labels
`nmap`, `curl`, `tcpdump`	Test and generate traffic

3.1.3. System Workflow

The system follows a sequential workflow that ensures efficient processing and accurate detection of network threats.

First, network traffic is continuously monitored and captured. The captured data is then analyzed by the Snort engine, which generates alerts based on rule matching. These alerts are passed to the preprocessing stage, where they are cleaned and formatted.

Next, the processed data is fed into the machine learning model for classification. The model predicts whether the activity is normal or malicious. Based on this classification, a risk score is assigned, and the event is categorized accordingly.

Finally, the results are displayed in the dashboard, allowing for real-time monitoring and response.

3.1.4. Software Module Description

The system is implemented as a set of software modules that operate together to achieve the overall functionality.

The detection module is responsible for running Snort and generating alerts from network traffic. The data processing module handles data cleaning, transformation, and preparation for machine learning input.

The classification module applies the trained Random Forest model to classify events. The risk assessment module computes the severity level of each alert. The visualization module provides a user interface for monitoring system outputs.

Each module is designed to operate independently while maintaining seamless integration within the overall system.

3.2. Modeling and Simulation

3.2.1. Dataset Description

The system utilizes the CICIDS2017 dataset, which is widely recognized as a benchmark dataset for intrusion detection research. This dataset contains a diverse set of network traffic records representing both normal activities and various types of cyberattacks[15].

The dataset includes multiple attack categories such as Distributed Denial of Service (DDoS), Port Scanning, Brute Force attacks, and infiltration attempts. Each record in the dataset is labeled, allowing supervised machine learning techniques to be applied effectively.

The selection of CICIDS2017 is based on its realistic traffic patterns and comprehensive coverage of modern attack scenarios, making it suitable for evaluating the performance of intrusion detection systems[15].

```
df["Attack_Type"] = df["Label"].map(attack_map)

print(df["Attack_Type"].value_counts())
print()
print("Missing:", df["Attack_Type"].isna().sum())
```

```
Attack_Type
BENIGN         2273097
DoS             380688
PortScan        158930
Brute Force      13835
Web Attack        2180
Rare Attack       2013
Name: count, dtype: int64

Missing: 0
```

Fig 3.2: CICIDS2017 Attack Type

3.2.2. Simulation/Prototype/Mathematical Model

The system is implemented as a prototype that simulates real-world intrusion detection scenarios. The core of the simulation is the Random Forest classification model, which is trained using labeled data from the dataset.

The model learns patterns associated with both normal and malicious traffic and uses this knowledge to classify new data instances. During simulation, input data is passed through the trained model, and predictions are generated in real time.

In addition to classification, a risk scoring mechanism is applied to assign a severity level to each detected event. This enhances the system’s ability to prioritize alerts effectively.

.

3.2.3. Deployment Steps

The deployment process involves running the system components in a coordinated manner to achieve real-time detection.

First, Snort is configured and executed to monitor network traffic and generate alerts. The generated alerts are then processed using the preprocessing module. The processed data is passed to the machine learning model, which performs classification and risk assessment.

Finally, the results are displayed through the dashboard interface, allowing users to monitor system activity continuously.

3.3. Hardware ImplementationThis section describes the implementation steps of the project. Some projects involve fabrication while others do not.

3.3.1. Implementation Steps

The implementation of the proposed system is carried out through a structured sequence of steps that integrate all system components into a unified workflow.

Initially, the operating environment is prepared by installing the required software tools, including the Snort intrusion detection system and Python libraries for data processing and machine learning. The CICIDS2017 dataset is then loaded and prepared for analysis through preprocessing and feature selection.

Table 3.2: stages for project

Stage	What Was Done	Purpose
Environment Setup	Ubuntu was prepared as the operating environment.	To provide a suitable platform for practical implementation.
Tool Installation	Snort, Python, and the required libraries for data processing and machine learning were installed.	To enable network analysis, model development, and system integration.
Data Preparation	The CICIDS2017 dataset was downloaded from Kaggle.	To use a realistic dataset for training and evaluation.
Preprocessing	The data was cleaned, encoded, and the most relevant features were selected.	To improve data quality before model training.
Model Training	A Random Forest model was trained using labeled data in Kaggle.	To build a classifier capable of identifying normal and malicious traffic.
Model Transfer	The trained model was saved and moved to the Ubuntu environment.	To use the trained model in the operational system.
System Integration	The model was integrated with the Snort alert processing pipeline.	To enable real-time classification of network alerts.
Network Monitoring	Snort was configured to monitor network traffic and generate alerts.	To detect suspicious activities during runtime.

Classification and Risk Scoring	Alerts were sent to the model for classification and risk evaluation.	To determine whether each event was benign or malicious.
Final Visualization	Results were displayed on a dashboard interface.	To support real-time monitoring of threats and system performance.

Following this, the Random Forest model is trained using labeled data. The trained model is then integrated with the alert processing pipeline to enable real-time classification of network events.

Snort is configured to monitor network traffic and generate alerts. These alerts are processed and passed to the machine learning model for classification and risk scoring. The final results are visualized through a dashboard interface, allowing users to monitor system performance and detected threats

Table of Project Implementation Stages

The system was implemented through a structured sequence of stages. First, the Ubuntu environment was prepared and the required tools, including Snort, Python, and supporting libraries, were installed. Next, the CICIDS2017 dataset was downloaded from Kaggle and processed through data cleaning, encoding, and feature selection. A Random Forest classifier was then trained on the labeled dataset in Kaggle and later transferred to the Ubuntu environment for deployment. After that, Snort was configured to monitor network traffic and generate alerts, which were then passed to the machine learning model for classification and risk scoring. Finally, the detected events and system outputs were presented through a dashboard for real-time monitoring and analysis.

3.3.2. Implementation Demo

The final system is demonstrated through a functional prototype that integrates all modules, including traffic monitoring, alert generation, classification, and visualization.

The dashboard interface displays detected alerts along with their assigned risk levels, enabling users to quickly identify critical threats. The system operates in real time, demonstrating its ability to process incoming data and generate meaningful insights.

The prototype validates the feasibility of the proposed design and confirms that the system components operate cohesively as intended.

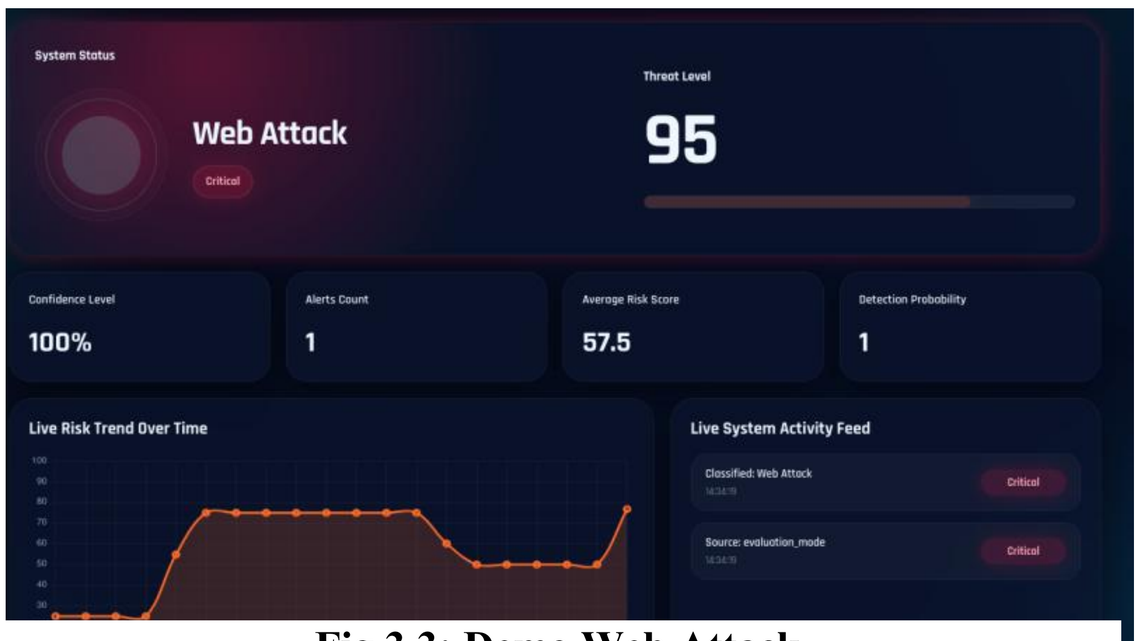


Fig 3.3: Demo Web Attack

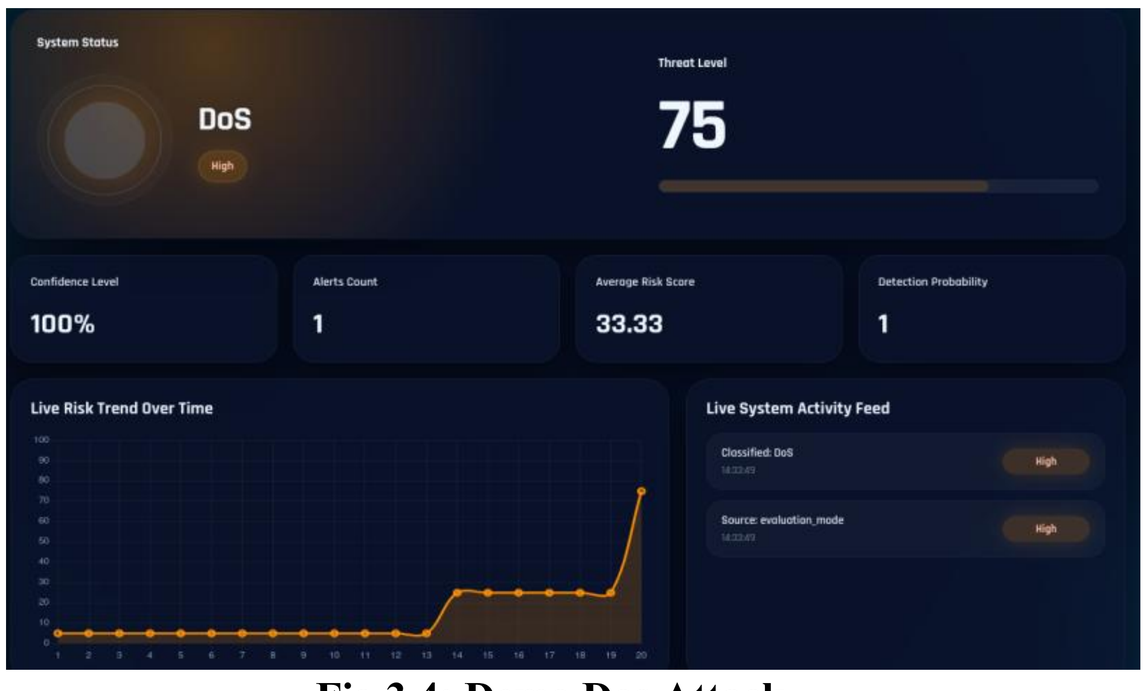


Fig 3.4: Demo Dos Attack

Chapter 4.

SYSTEM TESTING AND EVALUATION

4.1. Verification Test Description

This chapter presents the testing and evaluation process of the proposed AI-enhanced intrusion detection system. The evaluation phase was conducted to verify that the system performs correctly under different network conditions and successfully meets the functional and performance requirements defined earlier in this project.

The proposed system was tested using both normal and malicious network traffic to evaluate its ability to detect attacks, classify threats, reduce false alerts, and assign proper risk levels. In addition, system performance was measured in terms of detection accuracy, false-positive rate, processing speed, and hardware resource usage.

4.1.1. Testing Environment Setup

The testing environment was designed to simulate a realistic small-network monitoring scenario. The system was deployed on a Raspberry Pi platform running the proposed Snort and Random Forest hybrid architecture[17].

The software environment included Snort IDS for packet inspection, Python for data processing, and Scikit-Learn for the implementation of the Random Forest model. The CICIDS2017 dataset was used during the machine learning training and evaluation phases[15][21].

The system was connected to a local network environment where both normal and attack traffic could be generated and analyzed in real time.

4.1.2. Test Scenarios

Several testing scenarios were prepared to evaluate the proposed IDS under different traffic conditions. The scenarios included both legitimate traffic and multiple cyberattack categories commonly found in modern networks.

Normal traffic scenarios included standard web browsing, file transfers, and regular communication activities. Attack scenarios included port scanning, brute-force attacks, denial-of-service traffic, and suspicious abnormal behavior.

The purpose of these scenarios was to examine how effectively the proposed system could detect attacks, classify alerts, and assign appropriate risk levels.

Table 3.3:Scenarios Attack with Risk Level

Scenario	Traffic Type	Expected Risk Level
Scenario 1	Normal Network Traffic	Normal
Scenario 2	Port Scanning Attack	low
Scenario 3	Brute Force	Medium
Scenario 4	DoS/DDoS Traffic	High
Scenario 5	Web Attack	Critical

4.1.3. Functional Verification Tests

Functional verification tests were performed to confirm that all major components of the proposed IDS operate correctly.
The testing process verified that Snort successfully captures network packets and generates alerts in real time. The Random Forest model correctly classified alerts and assigned risk levels according to attack severity. The dashboard interface was also verified to ensure that alerts and classifications were displayed properly.

Table 3.4:Functional Verification Tests

Functional Requirement	Test Description	Result
Packet Capture	Real-time packet monitoring	Pass
Signature Detection	Snort alert generation	Pass
ML Classification	Random Forest classification	Pass
Risk Scoring	Alert severity assignment	Pass
Dashboard Visualization	Real-time alert display	Pass

4.2. Results and Analysis

This section presents the experimental results obtained from testing the proposed intrusion detection system. The results evaluate the effectiveness of combining Snort IDS with the Random Forest machine-learning model and the risk-scoring module.

The analysis focuses on detection performance, false alert reduction, resource utilization, and real-time monitoring capability.

4.2.1. Results

The proposed system was evaluated using a set of widely adopted metrics in cybersecurity and machine learning to assess the effectiveness of attack detection and the overall quality of classification. These metrics were selected to provide a comprehensive view of the model's performance, rather than relying on a single measure. In intrusion detection applications, this is especially important because the system must not only make correct predictions, but also minimize false alarms and detect malicious activities reliably.

Accuracy is one of the most common evaluation metrics and represents the ratio of correctly classified samples to the total number of samples. It provides a general indication of how often the model makes correct predictions. However, accuracy alone can be misleading in imbalanced datasets, where one class dominates the others. For this reason, additional metrics were used to obtain a more reliable assessment of performance.

Precision measures the proportion of instances predicted as attacks that were actually attacks. In other words, it reflects the model's ability to avoid false positives. A high precision value indicates that the system raises fewer unnecessary alerts, which is important in real-world security environments where excessive false alarms can reduce trust in the system.

Recall measures the proportion of actual attacks that were correctly identified by the model. It reflects the system's ability to detect malicious events without missing them. In cybersecurity, recall is particularly important because failing to detect an attack may lead to serious security consequences.

The F1-score is the harmonic mean of precision and recall, and it is used to balance the trade-off between these two metrics. It is especially useful when the dataset is imbalanced or when both false positives and false negatives must be considered carefully. A strong F1-score indicates that the model performs well in both detecting attacks and avoiding unnecessary alerts.

In addition, Support indicates the number of samples available for each class in the dataset. It helps explain how the evaluation metrics are distributed across different categories and provides context for interpreting the results. Support is particularly useful when comparing class-level performance, as some attack categories may contain more samples than others.
Overall, these metrics were chosen because they offer a detailed and balanced evaluation of the proposed system. Together, they provide insight into the classifier's correctness, reliability, and practical suitability for intrusion detection tasks.

	precision	recall	f1-score	support
0	1.00	1.00	1.00	454620
1	1.00	1.00	1.00	2767
2	1.00	1.00	1.00	76138
3	0.99	1.00	0.99	31786
4	0.93	0.77	0.84	402
5	1.00	0.96	0.98	436
accuracy			1.00	566149
macro avg	0.99	0.95	0.97	566149
weighted avg	1.00	1.00	1.00	566149

Fig 3.5:Results of model

4.2.2. Analysis

The evaluation results demonstrate that the proposed intrusion detection system achieved very strong classification performance across the test dataset. The overall accuracy reached 1.00, which indicates that the model correctly classified nearly all instances in the test set. This suggests that the combination of Snort-based alert generation and the Random Forest classifier was highly effective in distinguishing between normal and malicious traffic.

The class-level metrics show that most classes were identified with excellent precision, recall, and F1-score values. Classes 0, 1, and 2 achieved perfect scores across all three metrics, which means the model classified these categories without error in the test data. Class 3 also performed very well, with a precision of 0.99 and an F1-score of 0.99, indicating very few misclassifications. These results reflect the model's strong ability to learn the patterns associated with the major traffic categories.

Class 4 showed lower performance compared with the other classes, with a precision of 0.93, recall of 0.77, and F1-score of 0.84. This indicates that the model had more difficulty detecting this class consistently, and some instances were either misclassified or missed entirely. One possible explanation is that this class contains fewer samples than the dominant classes, as shown by its support value of 402, which can reduce the model's ability to generalize well for that category.

Class 5 also achieved strong results, with a precision of 1.00, recall of 0.96, and F1-score of 0.98. This means the model was highly accurate in identifying this class, although a small number of samples were not detected correctly. The macro average values, especially the recall of 0.95, show that performance was consistently strong across classes, while the weighted averages of 1.00 reflect the influence of the larger classes in the dataset.

Overall, the results indicate that the proposed system is highly reliable for network traffic classification and threat detection. The very high accuracy and F1-score values confirm that the

model generalizes well on unseen data, while the slight reduction in performance for class 4 highlights the impact of class imbalance and the difficulty of detecting rare categories. These findings support the suitability of the proposed approach for real-time intrusion detection and security monitoring

important segment of this report. This section must include the analysis of the results and how they are interpreted.

4.2.3. Risk Scoring Evaluation

The proposed system successfully implemented a real-time risk-scoring mechanism capable of classifying alerts into five severity levels: Normal, Low, Medium, High, and Critical.
This feature allows security analysts to prioritize dangerous threats and focus on incidents that require immediate attention.

Table 3.5: examples of attack classifications and their assigned risk levels.

Attack Type	Predicted Risk Level	Scoring rang
Normal Traffic	Normal	**5**
Port Scan	low	**25**
Brute Force Attack	Medium	**50**
DDoS Attack	High	**75**
Web Attack	Critical	**95**

4.3. Discussions

This section discusses the overall findings obtained from the experimental evaluation and explains the significance of the achieved results.

4.3.1. Discussion of Results

The experimental results demonstrate that integrating machine learning with Snort IDS significantly improves intrusion detection performance.
The Random Forest model enhanced the system's ability to classify suspicious traffic patterns and reduce unnecessary alerts. Compared to traditional signature-based detection alone, the proposed system showed improved adaptability against evolving network threats.
In addition, the real-time risk-scoring mechanism improved the efficiency of alert prioritization and provided more meaningful information to network administrators.
Overall, the results confirm that hybrid IDS architectures can provide stronger cybersecurity protection than conventional IDS systems.

4.3.2. System Strengths

The proposed system provides several important advantages.

First, the hybrid design combines the strengths of signature-based detection and machine-learning classification. This improves the system's ability to detect both known and unknown attacks.

Second, the Random Forest classifier reduces false alerts and improves detection reliability.

Third, the risk-scoring mechanism provides intelligent prioritization of threats, helping analysts focus on critical incidents.

Finally, the use of Raspberry Pi and open-source tools makes the system affordable, lightweight, and suitable for educational and small-scale deployment environments.

4.3.3. System Limitations

Despite the positive results, the proposed system still has several limitations.

The Random Forest model requires periodic retraining to maintain detection performance against new attack patterns. In addition, the overall system performance depends heavily on the quality and diversity of the training dataset.

The hardware limitations of Raspberry Pi may also affect performance under extremely high traffic volumes or complex enterprise-scale network environments.

Furthermore, the proposed system focuses mainly on intrusion detection and risk classification rather than automated response mechanisms.

Chapter 5.

CONCLUSIONS AND RECOMENDATIONS

5.1. Conclusions

This project presented the design and implementation of an AI-enhanced intrusion detection system that combines Snort IDS, a Random Forest machine-learning model, and a real-time risk-scoring mechanism. The proposed system was developed to address several limitations commonly found in traditional intrusion detection systems, particularly the inability to efficiently detect unknown attacks, the high number of false alerts, and the lack of intelligent alert prioritization.

The experimental results demonstrated that integrating machine learning with signature-based intrusion detection can significantly improve overall system performance. The proposed hybrid architecture achieved higher detection accuracy and lower false-positive rates compared to the traditional Snort-only approach. In addition, the risk-scoring mechanism improved the prioritization of security alerts and helped provide more meaningful threat analysis.

Another important outcome of this project is the successful deployment of the IDS on lightweight hardware using Raspberry Pi. This demonstrates that intelligent intrusion detection systems can be implemented without relying on expensive infrastructure or high-end processing hardware. The system maintained acceptable performance levels while operating in real-time monitoring conditions.

The project also confirmed that Random Forest is a practical and efficient machine-learning model for cybersecurity applications involving structured network traffic data. Its relatively low computational requirements make it suitable for embedded and resource-constrained environments.

Despite the positive results, several limitations remain. The machine-learning model's performance depends strongly on the quality and diversity of the training dataset.In addition, the system may require periodic retraining and rule updates to remain effective against newly emerging attack patterns. The Raspberry Pi platform may also face performance limitations under very large-scale network environments or extremely high traffic loads.

Future extensions to the project may include integrating deep-learning techniques, implementing automated response mechanisms, improving real-time feature extraction, and supporting cloud-based monitoring platforms. Additional enhancements may also involve online learning capabilities that allow the IDS to adapt dynamically to evolving network threats.

Overall, the proposed system successfully achieved the main objectives of the project and demonstrated that hybrid IDS architectures can provide effective, intelligent, and low-cost cybersecurity solutions for modern network environments[22][23].

5.2. Recommendations

If this project were to be developed again, several improvements could be considered during the design and implementation phases. One possible improvement would be using a larger and more diverse dataset during training in order to improve the model's generalization capability and increase detection reliability against modern attack variations.

Another improvement would involve testing the system under larger real-world traffic environments to evaluate scalability and long-term operational stability. Additional optimization of feature extraction and data preprocessing techniques may also improve processing speed and reduce system overhead.

During the development process, several important technical and practical lessons were learned. The project provided valuable experience in integrating cybersecurity tools with machine-learning technologies and demonstrated how software and hardware components can operate together within a real-time embedded system. The implementation process also improved understanding of network monitoring, packet inspection, attack classification, and system performance evaluation.

The project contributed significantly to practical engineering experience by combining multiple technical areas, including intrusion detection systems, machine learning, embedded systems, Linux administration, and network security analysis. It also highlighted the importance of balancing detection performance with hardware limitations and operational cost.

Several future extensions can further improve the proposed IDS. Future versions may include automated threat response capabilities, cloud-based dashboards, mobile alert notifications, and integration with Security Information and Event Management (SIEM) platforms. Deep-learning approaches such as LSTM or CNN models may also be explored to improve detection of complex attack behaviors.

In addition, the system may benefit from online learning techniques that continuously update the machine-learning model based on newly observed traffic patterns. Explainable AI techniques may also be integrated to improve transparency and help security analysts better understand model decisions.

Overall, the project established a strong foundation for future research and practical deployment of intelligent intrusion detection systems in lightweight and cost-efficient environments.

References.

[1] Mohale, V. Z., and Obagbuwa, I. C., 2025, “Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability”, Frontiers in Computer Science, Vol. 7, Article 1520741, 23 pages.

[2] Arnob, A. K. B., Chowdhury, R. R., Chaiti, N. A., Saha, S., and Roy, A., 2025, “A comprehensive systematic review of intrusion detection systems: emerging techniques, challenges, and future research directions”, Journal of Edge Computing, Vol. 4, Iss. 1, pp. 73–104.

[3] Sadargari Viharika and N. Alangudi Balaji, 2025, “Enhancing Intrusion Detection and Cloud Security by Integrating Snort with Advanced AI Techniques for Improved Accuracy and Threat Mitigation”, Journal of Information Systems Engineering and Management, Vol. 10, No. 24(s), pp. 628–632.

[4] Otoum, S., Alsharif, A., and Kachouie, R., 2023, “Intelligent Intrusion Detection System Snort and SVM”, Revue d’Intelligence Artificielle, Vol. 37, No. 6, pp. 1629–1635.

[5] Kaushik, S., Gupta, P., and Singh, R., 2025, “Robust machine learning based intrusion detection system”, Scientific Reports, Vol. 15, pp. 1–12.

[6] Hindy, H., Brosset, D., Bayne, E., Seeam, A., Tachtatzis, C., Atkinson, R., and Bellekens, X., 2020, “A taxonomy of network threats and the effect of current datasets on intrusion detection systems”, IEEE Access, Vol. 8, pp. 104650–104675.

[7] Pendyala, G. S., Mukherjee, T., and Shetty, N. P., 2025, “Intrusion detection using graph inspired insights”, International Journal of Computers and Applications, Vol. 47, No. 12, pp. 1076–1105.

[8] Chavan, P. V., and Alone, N. V., 2025, “Optimizing Intrusion Detection with Random Forest: A High-Accuracy Approach using CIC-IDS 2017”, International Journal of Computer Applications, Vol. 187, No. 3, pp. 17–24.

[9] Wang, X., Li, Y., and Chen, H., 2025, “Explainable artificial intelligence models in intrusion detection systems”, Engineering Applications of Artificial Intelligence, Vol. 130, pp. 107–124.

[10] Liu, Y., Zhang, J., and Huang, K., 2024, “Hybrid intrusion detection system combining signature-based Snort rules with machine learning-based anomaly detection”, Computers & Security, Vol. 133, pp. 209–224.

[11] Abedin, M. N., Nessa, A., and Rahman, M. A., 2023, “A hybrid intrusion detection framework integrating Snort with machine learning techniques for improved network security”, International Journal of Information Security Science, Vol. 12, No. 4, pp. 215–232.

[12] Ghazi, D. S., Hamid, H. S., Zaiter, M. J., and Behadili, A. S. G., 2024, “Snort versus Suricata in intrusion detection”, Iraqi Journal of Information and Communication Technology, Vol. 7, No. 2, pp. 73–88.

[13] Nagallo, J. L. O., Baluarte, J. Y. S., Cailao, R. J. D., Sible, L. I., and Mag-isa, J. C., 2025, “Comparative study of Random Forest (ML) and Snort (signature) IDS: detection performance (accuracy and false alarms) and operational overhead in public cloud deployments”, Cognizance Journal of Multidisciplinary Studies, Vol. 5, Issue 10, pp. 389–394.

[14] Shahid, N., Anees, A., Shah, M. A., and Khan, S., 2024, “Enhanced intrusion detection systems performance with UNSW-NB15 data analysis”, Algorithms, Vol. 17, No. 2, Article 64.

[15] Azhar, M., Perveen, S., Iqbal, A., and Lee, B., 2024, “IDRandom-Forest: Advanced Random Forest for real-time intrusion detection”, IEEE Access, Vol. 12, pp. 1–20

[16] Vadher, P., Siddiqui, M., 2020, "Snort IDPS using Raspberry Pi 4", International Journal of Engineering Research & Technology, Vol. 9, Issue 7, pp. 1234-1240.

[17] Karahan, O., Kaya, B., 2020, "Raspberry Pi Firewall and Intrusion Detection System", Journal of Intelligent Systems Theory and Applications, Vol. 3, No. 2, pp. 21-24.

[18] Albasheer, H., Siraj, M.M., Mubarakali, A., Elsir Tayfour, O., et al., 2022, "Cyber-attack prediction based on network intrusion detection systems for alert correlation techniques: a survey", Sensors, Vol. 22, No. 4, Article 1494. [web:22]

[19] Khan, M.H., 2019, "Building Affordable IDS using Snort and Elastic Stack on Raspberry Pi", M.Sc. Thesis, Chalmers University, 92 pages.

[20] Anonymous, 2025, "Intrusion Detection System using Raspberry Pi for IoT Devices", International Journal for Research in Applied Science & Engineering Technology, Vol. 13, No. 4, pp. 123-130.

[21] Sommer, R., and Paxson, V., 2010, "Outside the closed world: On using machine learning for network intrusion detection", *IEEE Symposium on Security and Privacy*, pp. 305–316.

[22] Tsai, C. F., Hsu, Y. F., Lin, C. Y., and Lin, W. Y., 2009, "Intrusion detection by machine learning: A review", Expert Systems with Applications, Vol. 36, No. 10, pp. 11994–12000.

[23] Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., and Venkatraman, S., 2019, "Robust intelligent malware detection using deep learning", *IEEE Access*, Vol. 7, pp. 46717–46738.